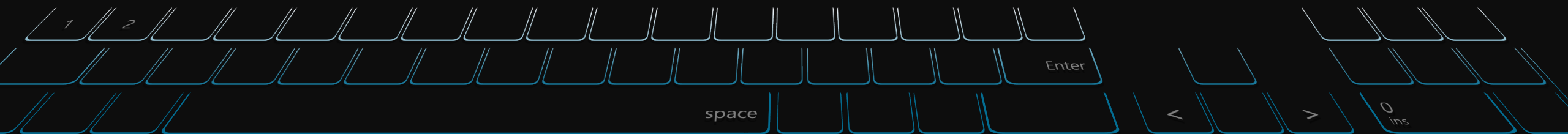


08 데이터 집계와 그룹 연산



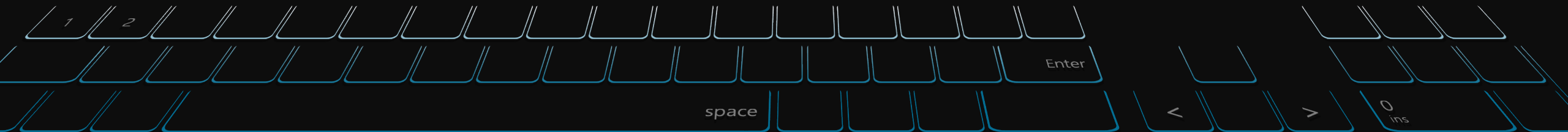


강의 내용

Contents of Lecture

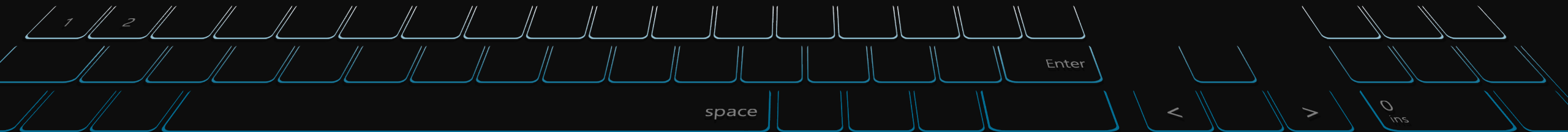
기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!



1부: 데이터 집계와 그룹 연산

- 1) Groupby 공학
- 2) 데이터 집계
- 3) Apply 함수
- 4) 피벗테이블 및 교차표





데이터 집계 및 그룹 연산

Data Aggregation and Group Operations

- 1) 데이터 세트를 분류하고 각 그룹에 함수를 적용하는 것은 종종 데이터 분석 워크플로의 중요한 구성 요소
 - 2) 그룹 통계 또는 피벗 테이블을 계산
 - > 데이터 세트를 로드, 병합 및 준비한 후 보고 또는 시각화 위함
 - 3) pandas는 유연한 groupby 인터페이스를 제공하여, 자연스러운 방식으로 데이터 세트를 슬라이스 및 요약
 - 4) 관계형 데이터베이스와 SQL("구조화된 쿼리 언어"의 약자)이 인기 있는 한 가지 이유
 - > 데이터를 쉽게 결합, 필터링, 변환 및 집계할 수 있기 때문
 - > 그러나 SQL과 같은 쿼리 언어는 수행할 수 있는 그룹 작업의 종류에 다소 제약이 존재
 - > 하지만, Python과 pandas를 통해 pandas 객체 또는 NumPy 배열을 허용하는 모든 함수를 활용하여, 매우 복잡한 그룹 작업을 수행할 수 있음
- 하나 이상의 키(함수, 배열 또는 DataFrame 열 이름의 형태로)를 사용하여 pandas 객체를 조각으로 나눔
 - 개수, 평균 또는 표준 편차와 같은 그룹 요약 통계 또는 사용자 정의 함수 계산
 - 그룹 내 변환 또는 정규화, 선형 회귀, 순위 또는 하위 집합 선택과 같은 기타 조작 적용
 - 피벗 테이블 및 교차 테이블 계산
 - 위수 분석 및 기타 통계 그룹 분석 수행

groupby의 특별한 사용 사례인 시계열 데이터의 집계인 리샘플링은 다음 수업 때 다룰 예정

10.1 GroupBy Mechanics

Hadley Wickham, an author of many popular packages for the R programming language, coined the term *split-apply-combine* for describing group operations. In the first stage of the process, data contained in a pandas object, whether a Series, DataFrame, or otherwise, is *split* into groups based on one or more *keys* that you provide. The splitting is performed on a particular axis of an object. For example, a DataFrame can be grouped on its rows (*axis=0*) or its columns (*axis=1*). Once this is done, a function is *applied* to each group, producing a new value. Finally, the results of all those function applications are *combined* into a result object. The form of the resulting object will usually depend on what's being done to the data. See [Figure 10-1](#) for a mockup of a simple group aggregation.

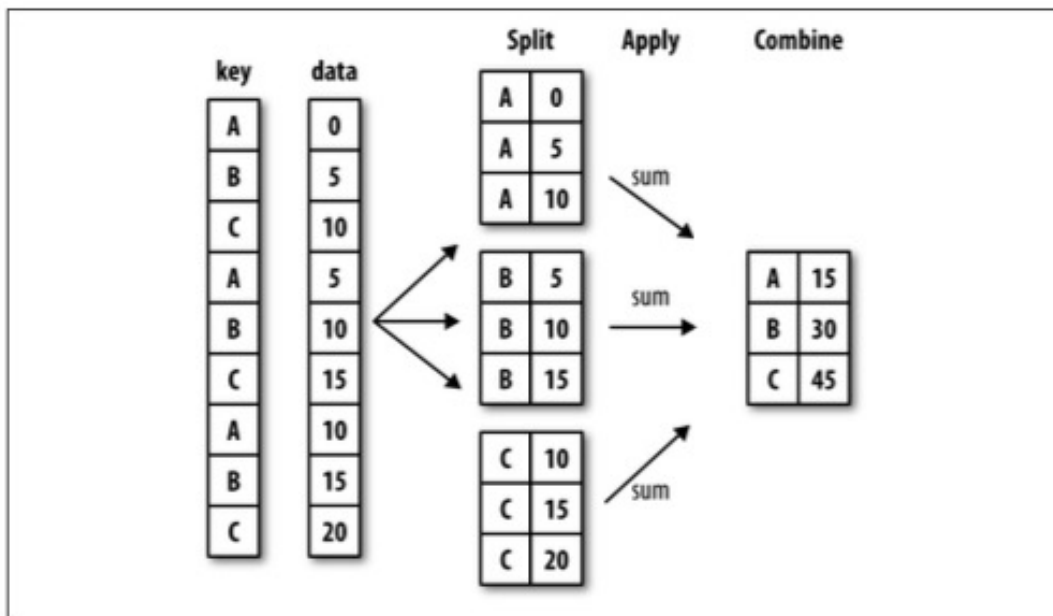


Figure 10-1. Illustration of a group aggregation

Each grouping key can take many forms, and the keys do not have to be all of the same type:

- A list or array of values that is the same length as the axis being grouped
- A value indicating a column name in a DataFrame
- A dict or Series giving a correspondence between the values on the axis being grouped and the group names
- A function to be invoked on the axis index or the individual labels in the index

Note that the latter three methods are shortcuts for producing an array of values to be used to split up the object. Don't worry if this all seems abstract. Throughout this chapter, I will give many examples of all these methods. To get started, here is a small tabular dataset as a DataFrame:

```
In [10]: df = pd.DataFrame({'key1': ['a', 'a', 'b', 'b', 'a'],
...:                        'key2': ['one', 'two', 'one', 'two', 'one'],
...:                        'data1': np.random.randn(5),
...:                        'data2': np.random.randn(5)})
```

```
In [11]: df
Out[11]:
```

	data1	data2	key1	key2
0	-0.204708	1.393406	a	one
1	0.478943	0.092908	a	two
2	-0.519439	0.281746	b	one
3	-0.555730	0.769023	b	two
4	1.965781	1.246435	a	one



Suppose you wanted to compute the mean of the `data1` column using the labels from `key1`. There are a number of ways to do this. One is to access `data1` and call `groupby` with the column (a Series) at `key1`:

```
In [12]: grouped = df['data1'].groupby(df['key1'])

In [13]: grouped
Out[13]: <pandas.core.groupby.SeriesGroupBy object at 0x7faa31537390>
```

This `grouped` variable is now a *GroupBy* object. It has not actually computed anything yet except for some intermediate data about the group key `df['key1']`. The idea is that this object has all of the information needed to then apply some operation to each of the groups. For example, to compute group means we can call the *GroupBy*'s `mean` method:

```
In [14]: grouped.mean()
Out[14]:
key1
a      0.746672
b     -0.537585
Name: data1, dtype: float64
```

Later, I'll explain more about what happens when you call `.mean()`. The important thing here is that the data (a Series) has been aggregated according to the group key, producing a new Series that is now indexed by the unique values in the `key1` column.

The result index has the name `'key1'` because the DataFrame column `df['key1']` did.

If instead we had passed multiple arrays as a list, we'd get something different:

```
In [15]: means = df['data1'].groupby([df['key1'], df['key2']]).mean()

In [16]: means
Out[16]:
key1 key2
a     one    0.880536
      two    0.478943
b     one   -0.519439
      two   -0.555730
Name: data1, dtype: float64
```

Here we grouped the data using two keys, and the resulting Series now has a hierarchical index consisting of the unique pairs of keys observed:

```
In [17]: means.unstack()
Out[17]:
key2      one      two
key1
a      0.880536  0.478943
b     -0.519439 -0.555730
```

In this example, the group keys are all Series, though they could be any arrays of the right length:

```
In [18]: states = np.array(['Ohio', 'California', 'California', 'Ohio', 'Ohio'])
In [19]: years = np.array([2005, 2005, 2006, 2005, 2006])

In [20]: df['data1'].groupby([states, years]).mean()
Out[20]:
California 2005    0.478943
           2006   -0.519439
Ohio       2005   -0.380219
           2006    1.965781
Name: data1, dtype: float64
```



Frequently the grouping information is found in the same DataFrame as the data you want to work on. In that case, you can pass column names (whether those are strings, numbers, or other Python objects) as the group keys:

```
In [21]: df.groupby('key1').mean()
```

```
Out[21]:
```

	data1	data2
key1		
a	0.746672	0.910916
b	-0.537585	0.525384

```
In [22]: df.groupby(['key1', 'key2']).mean()
```

```
Out[22]:
```

		data1	data2
key1	key2		
a	one	0.880536	1.319920
	two	0.478943	0.092908
b	one	-0.519439	0.281746
	two	-0.555730	0.769023

You may have noticed in the first case `df.groupby('key1').mean()` that there is no `key2` column in the result. Because `df['key2']` is not numeric data, it is said to be a *nuisance column*, which is therefore excluded from the result. By default, all of the numeric columns are aggregated, though it is possible to filter down to a subset, as you'll see soon.

You may have noticed in the first case `df.groupby('key1').mean()` that there is no `key2` column in the result. Because `df['key2']` is not numeric data, it is said to be a *nuisance column*, which is therefore excluded from the result. By default, all of the numeric columns are aggregated, though it is possible to filter down to a subset, as you'll see soon.

Regardless of the objective in using `groupby`, a generally useful `GroupBy` method is `size`, which returns a Series containing group sizes:

```
In [23]: df.groupby(['key1', 'key2']).size()
```

```
Out[23]:
```

key1	key2	
a	one	2
	two	1
b	one	1
	two	1

dtype: int64

Take note that any missing values in a group key will be excluded from the result.



Groupby 공학 > Iterating Over Groups

GroupBy Mechanics

Iterating Over Groups

The GroupBy object supports iteration, generating a sequence of 2-tuples containing the group name along with the chunk of data. Consider the following:

```
In [24]: for name, group in df.groupby('key1'):
.....:     print(name)
.....:     print(group)
.....:
```

	data1	data2	key1	key2
a				
0	-0.204708	1.393406	a	one
1	0.478943	0.092908	a	two
4	1.965781	1.246435	a	one
b				
2	-0.519439	0.281746	b	one
3	-0.555730	0.769023	b	two

In the case of multiple keys, the first element in the tuple will be a tuple of key values:

```
In [25]: for (k1, k2), group in df.groupby(['key1', 'key2']):
.....:     print((k1, k2))
.....:     print(group)
.....:
```

	data1	data2	key1	key2
('a', 'one')				
0	-0.204708	1.393406	a	one
4	1.965781	1.246435	a	one
('a', 'two')				
1	0.478943	0.092908	a	two
('b', 'one')				
2	-0.519439	0.281746	b	one
('b', 'two')				
3	-0.55573	0.769023	b	two

Of course, you can choose to do whatever you want with the pieces of data. A recipe you may find useful is computing a dict of the data pieces as a one-liner:

```
In [26]: pieces = dict(list(df.groupby('key1')))

In [27]: pieces['b']
Out[27]:
```

	data1	data2	key1	key2
2	-0.519439	0.281746	b	one
3	-0.555730	0.769023	b	two

By default groupby groups on axis=0, but you can group on any of the other axes. For example, we could group the columns of our example df here by dtype like so:

```
In [28]: df.dtypes
Out[28]:
```

	data1	data2	key1	key2
data1	float64			
data2		float64		
key1			object	
key2				object
dtype:	object			

```
In [29]: grouped = df.groupby(df.dtypes, axis=1)
```

We can print out the groups like so:

```
In [30]: for dtype, group in grouped:
.....:     print(dtype)
.....:     print(group)
.....:
```

	data1	data2
float64		
0	-0.204708	1.393406
1	0.478943	0.092908
2	-0.519439	0.281746
3	-0.555730	0.769023
4	1.965781	1.246435
object		
key1		
0	a	one
1	a	two
2	b	one
3	b	two
4	a	one



Groupby 공학 > Selecting a Column or Subset of Columns

GroupBy Mechanics

Selecting a Column or Subset of Columns

Indexing a GroupBy object created from a DataFrame with a column name or array of column names has the effect of column subsetting for aggregation. This means that:

```
df.groupby('key1')['data1']  
df.groupby('key1')[['data2']]
```

are syntactic sugar for:

```
df['data1'].groupby(df['key1'])  
df[['data2']].groupby(df['key1'])
```

Especially for large datasets, it may be desirable to aggregate only a few columns. For example, in the preceding dataset, to compute means for just the data2 column and get the result as a DataFrame, we could write:

```
In [31]: df.groupby(['key1', 'key2'])['data2'].mean()  
Out[31]:
```

		data2
key1	key2	
a	one	1.319920
	two	0.092908
b	one	0.281746
	two	0.769023

The object returned by this indexing operation is a grouped DataFrame if a list or array is passed or a grouped Series if only a single column name is passed as a scalar:

```
In [32]: s_grouped = df.groupby(['key1', 'key2'])['data2']  
  
In [33]: s_grouped  
Out[33]: <pandas.core.groupby.SeriesGroupBy object at 0x7faa30c78da0>  
  
In [34]: s_grouped.mean()  
Out[34]:
```

key1	key2	
a	one	1.319920
	two	0.092908
b	one	0.281746
	two	0.769023

Name: data2, dtype: float64

Groupby 공학 > Grouping with Dicts and Series

GroupBy Mechanics

Grouping with Dicts and Series

Grouping information may exist in a form other than an array. Let's consider another example DataFrame:

```
In [35]: people = pd.DataFrame(np.random.randn(5, 5),
.....:                        columns=['a', 'b', 'c', 'd', 'e'],
.....:                        index=['Joe', 'Steve', 'Wes', 'Jim', 'Travis'])
```

```
In [36]: people.iloc[2:3, [1, 2]] = np.nan # Add a few NA values
```

```
In [37]: people
```

```
Out[37]:
```

	a	b	c	d	e
Joe	1.007189	-1.296221	0.274992	0.228913	1.352917
Steve	0.886429	-2.001637	-0.371843	1.669025	-0.438570
Wes	-0.539741	NaN	NaN	-1.021228	-0.577087
Jim	0.124121	0.302614	0.523772	0.000940	1.343810
Travis	-0.713544	-0.831154	-2.370232	-1.860761	-0.860757

Now, suppose I have a group correspondence for the columns and want to sum together the columns by group:

```
In [38]: mapping = {'a': 'red', 'b': 'red', 'c': 'blue',
.....:              'd': 'blue', 'e': 'red', 'f': 'orange'}
```

Now, you could construct an array from this dict to pass to groupby, but instead we can just pass the dict (I included the key 'f' to highlight that unused grouping keys are OK):

```
In [39]: by_column = people.groupby(mapping, axis=1)
```

```
In [40]: by_column.sum()
```

```
Out[40]:
```

	blue	red
Joe	0.503905	1.063885
Steve	1.297183	-1.553778
Wes	-1.021228	-1.116829
Jim	0.524712	1.770545
Travis	-4.230992	-2.405455

The same functionality holds for Series, which can be viewed as a fixed-size mapping:

```
In [41]: map_series = pd.Series(mapping)
```

```
In [42]: map_series
```

```
Out[42]:
```

a	red
b	red
c	blue
d	blue
e	red
f	orange

dtype: object

```
In [43]: people.groupby(map_series, axis=1).count()
```

```
Out[43]:
```

	blue	red
Joe	2	3
Steve	2	3
Wes	1	2
Jim	2	3
Travis	2	3



Groupby 공학 > Grouping with Functions

GroupBy Mechanics

Grouping with Functions

Using Python functions is a more generic way of defining a group mapping compared with a dict or Series. Any function passed as a group key will be called once per index value, with the return values being used as the group names. More concretely, consider the example DataFrame from the previous section, which has people's first names as index values. Suppose you wanted to group by the length of the names; while you could compute an array of string lengths, it's simpler to just pass the `len` function:

```
In [44]: people.groupby(len).sum()
Out[44]:
```

	a	b	c	d	e
3	0.591569	-0.993608	0.798764	-0.791374	2.119639
5	0.886429	-2.001637	-0.371843	1.669025	-0.438570
6	-0.713544	-0.831154	-2.370232	-1.860761	-0.860757

Mixing functions with arrays, dicts, or Series is not a problem as everything gets converted to arrays internally:

```
In [45]: key_list = ['one', 'one', 'one', 'two', 'two']

In [46]: people.groupby([len, key_list]).min()
Out[46]:
```

	a	b	c	d	e
3 one	-0.539741	-1.296221	0.274992	-1.021228	-0.577087
two	0.124121	0.302614	0.523772	0.000940	1.343810
5 one	0.886429	-2.001637	-0.371843	1.669025	-0.438570
6 two	-0.713544	-0.831154	-2.370232	-1.860761	-0.860757



Groupby 공학 > Grouping by Index Levels

GroupBy Mechanics

Grouping by Index Levels

A final convenience for hierarchically indexed datasets is the ability to aggregate using one of the levels of an axis index. Let's look at an example:

```
In [47]: columns = pd.MultiIndex.from_arrays([[ 'US', 'US', 'US', 'JP', 'JP'],
.....:                                     [1, 3, 5, 1, 3]],
.....:                                     names=['cty', 'tenor'])
```

```
In [48]: hier_df = pd.DataFrame(np.random.randn(4, 5), columns=columns)
```

```
In [49]: hier_df
```

```
Out[49]:
```

	US			JP	
cty					
tenor	1	3	5	1	3
0	0.560145	-1.265934	0.119827	-1.063512	0.332883
1	-2.359419	-0.199543	-1.541996	-0.970736	-1.307030
2	0.286350	0.377984	-0.753887	0.331286	1.349742
3	0.069877	0.246674	-0.011862	1.004812	1.327195

To group by level, pass the level number or name using the level keyword:

```
In [50]: hier_df.groupby(level='cty', axis=1).count()
```

```
Out[50]:
```

cty	JP	US
0	2	3
1	2	3
2	2	3
3	2	3



10.2 Data Aggregation

Aggregations refer to any data transformation that produces scalar values from arrays. The preceding examples have used several of them, including `mean`, `count`, `min`, and `sum`. You may wonder what is going on when you invoke `mean()` on a `GroupBy` object. Many common aggregations, such as those found in [Table 10-1](#), have optimized implementations. However, you are not limited to only this set of methods.

Table 10-1. Optimized groupby methods

Function name	Description
<code>count</code>	Number of non-NA values in the group
<code>sum</code>	Sum of non-NA values
<code>mean</code>	Mean of non-NA values
<code>median</code>	Arithmetic median of non-NA values
<code>std, var</code>	Unbiased ($n - 1$ denominator) standard deviation and variance
<code>min, max</code>	Minimum and maximum of non-NA values
<code>prod</code>	Product of non-NA values
<code>first, last</code>	First and last non-NA values

You can use aggregations of your own devising and additionally call any method that is also defined on the grouped object. For example, you might recall that `quantile` computes sample quantiles of a `Series` or a `DataFrame`'s columns.

While `quantile` is not explicitly implemented for `GroupBy`, it is a `Series` method and thus available for use. Internally, `GroupBy` efficiently slices up the `Series`, calls `piece.quantile(0.9)` for each piece, and then assembles those results together into the result object:

```
In [51]: df
Out[51]:
```

	data1	data2	key1	key2
0	-0.204708	1.393406	a	one
1	0.478943	0.092908	a	two
2	-0.519439	0.281746	b	one
3	-0.555730	0.769023	b	two
4	1.965781	1.246435	a	one

```
In [52]: grouped = df.groupby('key1')

In [53]: grouped['data1'].quantile(0.9)
Out[53]:
key1
a    1.668413
b   -0.523068
Name: data1, dtype: float64
```

To use your own aggregation functions, pass any function that aggregates an array to the `aggregate` or `agg` method:

```
In [54]: def peak_to_peak(arr):
.....:     return arr.max() - arr.min()

In [55]: grouped.agg(peak_to_peak)
Out[55]:
```

	data1	data2
key1		
a	2.170488	1.300498
b	0.036292	0.487276



데이터 집계

Data Aggregation

You may notice that some methods like `describe` also work, even though they are not aggregations, strictly speaking:

```
In [56]: grouped.describe()
Out[56]:
```

		data1						
		count	mean	std	min	25%	50%	75%
key1								
a		3.0	0.746672	1.109736	-0.204708	0.137118	0.478943	1.222362
b		2.0	-0.537585	0.025662	-0.555730	-0.546657	-0.537585	-0.528512
		data2						
		max	count	mean	std	min	25%	50%
key1								
a		1.965781	3.0	0.910916	0.712217	0.092908	0.669671	1.246435
b		-0.519439	2.0	0.525384	0.344556	0.281746	0.403565	0.525384
		75%	max					
key1								
a		1.319920	1.393406					
b		0.647203	0.769023					

I will explain in more detail what has happened here in [Section 10.3, “Apply: General split-apply-combine,”](#) on page 302.



Custom aggregation functions are generally much slower than the optimized functions found in [Table 10-1](#). This is because there is some extra overhead (function calls, data rearrangement) in constructing the intermediate group data chunks.



데이터 집계 > Column-Wise and Multiple Function Application

Data Aggregation

Column-Wise and Multiple Function Application

Let's return to the tipping dataset from earlier examples. After loading it with `read_csv`, we add a tipping percentage column `tip_pct`:

```
In [57]: tips = pd.read_csv('examples/tips.csv')

# Add tip percentage of total bill
In [58]: tips['tip_pct'] = tips['tip'] / tips['total_bill']

In [59]: tips[:6]
Out[59]:
```

	total_bill	tip	smoker	day	time	size	tip_pct
0	16.99	1.01	No	Sun	Dinner	2	0.059447
1	10.34	1.66	No	Sun	Dinner	3	0.160542
2	21.01	3.50	No	Sun	Dinner	3	0.166587
3	23.68	3.31	No	Sun	Dinner	2	0.139780
4	24.59	3.61	No	Sun	Dinner	4	0.146808
5	25.29	4.71	No	Sun	Dinner	4	0.186240

As you've already seen, aggregating a Series or all of the columns of a DataFrame is a matter of using `aggregate` with the desired function or calling a method like `mean` or `std`. However, you may want to aggregate using a different function depending on the column, or multiple functions at once. Fortunately, this is possible to do, which I'll illustrate through a number of examples. First, I'll group the tips by day and smoker:

```
In [60]: grouped = tips.groupby(['day', 'smoker'])
```

Note that for descriptive statistics like those in [Table 10-1](#), you can pass the name of the function as a string:

```
In [61]: grouped_pct = grouped['tip_pct']

In [62]: grouped_pct.agg('mean')
Out[62]:
```

day	smoker	
Fri	No	0.151650
	Yes	0.174783
Sat	No	0.158048
	Yes	0.147906
Sun	No	0.160113
	Yes	0.187250
Thur	No	0.160298
	Yes	0.163863

Name: tip_pct, dtype: float64

If you pass a list of functions or function names instead, you get back a DataFrame with column names taken from the functions:

```
In [63]: grouped_pct.agg(['mean', 'std', 'peak_to_peak'])
Out[63]:
```

		mean	std	peak_to_peak
day	smoker			
Fri	No	0.151650	0.028123	0.067349
	Yes	0.174783	0.051293	0.159925
Sat	No	0.158048	0.039767	0.235193
	Yes	0.147906	0.061375	0.290095
Sun	No	0.160113	0.042347	0.193226
	Yes	0.187250	0.154134	0.644685
Thur	No	0.160298	0.038774	0.193350
	Yes	0.163863	0.039389	0.151240

Here we passed a list of aggregation functions to `agg` to evaluate independently on the data groups.



데이터 집계 > Column-Wise and Multiple Function Application

Data Aggregation

You don't need to accept the names that GroupBy gives to the columns; notably, lambda functions have the name '<lambda>', which makes them hard to identify (you can see for yourself by looking at a function's `__name__` attribute). Thus, if you pass a list of (name, function) tuples, the first element of each tuple will be used as the DataFrame column names (you can think of a list of 2-tuples as an ordered mapping):

```
In [64]: grouped_pct.agg([('foo', 'mean'), ('bar', np.std)])
```

```
Out[64]:
```

		foo	bar
day	smoker		
Fri	No	0.151650	0.028123
	Yes	0.174783	0.051293
Sat	No	0.158048	0.039767
	Yes	0.147906	0.061375
Sun	No	0.160113	0.042347
	Yes	0.187250	0.154134
Thur	No	0.160298	0.038774
	Yes	0.163863	0.039389

With a DataFrame you have more options, as you can specify a list of functions to apply to all of the columns or different functions per column. To start, suppose we wanted to compute the same three statistics for the `tip_pct` and `total_bill` columns:

```
In [65]: functions = ['count', 'mean', 'max']
```

```
In [66]: result = grouped['tip_pct', 'total_bill'].agg(functions)
```

```
In [67]: result
```

```
Out[67]:
```

		tip_pct			total_bill		
		count	mean	max	count	mean	max
Fri	No	4	0.151650	0.187735	4	18.420000	22.75
	Yes	15	0.174783	0.263480	15	16.813333	40.17
Sat	No	45	0.158048	0.291990	45	19.661778	48.33
	Yes	42	0.147906	0.325733	42	21.276667	50.81
Sun	No	57	0.160113	0.252672	57	20.506667	48.17
	Yes	19	0.187250	0.710345	19	24.120000	45.35
Thur	No	45	0.160298	0.266312	45	17.113111	41.19
	Yes	17	0.163863	0.241255	17	19.190588	43.11



데이터 집계 > Column-Wise and Multiple Function Application

Data Aggregation

As you can see, the resulting DataFrame has hierarchical columns, the same as you would get aggregating each column separately and using concat to glue the results together using the column names as the keys argument:

```
In [68]: result['tip_pct']
Out[68]:
```

		count	mean	max
day	smoker			
Fri	No	4	0.151650	0.187735
	Yes	15	0.174783	0.263480
Sat	No	45	0.158048	0.291990
	Yes	42	0.147906	0.325733
Sun	No	57	0.160113	0.252672
	Yes	19	0.187250	0.710345
Thur	No	45	0.160298	0.266312
	Yes	17	0.163863	0.241255

As before, a list of tuples with custom names can be passed:

```
In [69]: ftuples = [('Durchschnitt', 'mean'), ('Abweichung', np.var)]
In [70]: grouped['tip_pct', 'total_bill'].agg(ftuples)
Out[70]:
```

		tip_pct		total_bill	
		Durchschnitt	Abweichung	Durchschnitt	Abweichung
day	smoker				
Fri	No	0.151650	0.000791	18.420000	25.596333
	Yes	0.174783	0.002631	16.813333	82.562438
Sat	No	0.158048	0.001581	19.661778	79.908965
	Yes	0.147906	0.003767	21.276667	101.387535
Sun	No	0.160113	0.001793	20.506667	66.099980
	Yes	0.187250	0.023757	24.120000	109.046044
Thur	No	0.160298	0.001503	17.113111	59.625081
	Yes	0.163863	0.001551	19.190588	69.808518

Now, suppose you wanted to apply potentially different functions to one or more of the columns. To do this, pass a dict to agg that contains a mapping of column names to any of the function specifications listed so far:

```
In [71]: grouped.agg({'tip' : np.max, 'size' : 'sum'})
Out[71]:
```

		tip	size
day	smoker		
Fri	No	3.50	9
	Yes	4.73	31
Sat	No	9.00	115
	Yes	10.00	104
Sun	No	6.00	167
	Yes	6.50	49
Thur	No	6.70	112
	Yes	5.00	40

```
In [72]: grouped.agg({'tip_pct' : ['min', 'max', 'mean', 'std'],
.....:                  'size' : 'sum'})
Out[72]:
```

		tip_pct				size
		min	max	mean	std	sum
day	smoker					
Fri	No	0.120385	0.187735	0.151650	0.028123	9
	Yes	0.103555	0.263480	0.174783	0.051293	31
Sat	No	0.056797	0.291990	0.158048	0.039767	115
	Yes	0.035638	0.325733	0.147906	0.061375	104
Sun	No	0.059447	0.252672	0.160113	0.042347	167
	Yes	0.065660	0.710345	0.187250	0.154134	49
Thur	No	0.072961	0.266312	0.160298	0.038774	112
	Yes	0.090014	0.241255	0.163863	0.039389	40

A DataFrame will have hierarchical columns only if multiple functions are applied to at least one column.



데이터 집계 > *Returning Aggregated Data Without Row Indexes*

Data Aggregation

Returning Aggregated Data Without Row Indexes

In all of the examples up until now, the aggregated data comes back with an index, potentially hierarchical, composed from the unique group key combinations. Since this isn't always desirable, you can disable this behavior in most cases by passing `as_index=False` to `groupby`:

```
In [73]: tips.groupby(['day', 'smoker'], as_index=False).mean()
```

```
Out[73]:
```

	day	smoker	total_bill	tip	size	tip_pct
0	Fri	No	18.420000	2.812500	2.250000	0.151650
1	Fri	Yes	16.813333	2.714000	2.066667	0.174783
2	Sat	No	19.661778	3.102889	2.555556	0.158048
3	Sat	Yes	21.276667	2.875476	2.476190	0.147906
4	Sun	No	20.506667	3.167895	2.929825	0.160113
5	Sun	Yes	24.120000	3.516842	2.578947	0.187250
6	Thur	No	17.113111	2.673778	2.488889	0.160298
7	Thur	Yes	19.190588	3.030000	2.352941	0.163863

Of course, it's always possible to obtain the result in this format by calling `reset_index` on the result. Using the `as_index=False` method avoids some unnecessary computations.



10.3 Apply: General split-apply-combine

The most general-purpose GroupBy method is `apply`, which is the subject of the rest of this section. As illustrated in **Figure 10-2**, `apply` splits the object being manipulated into pieces, invokes the passed function on each piece, and then attempts to concatenate the pieces together.

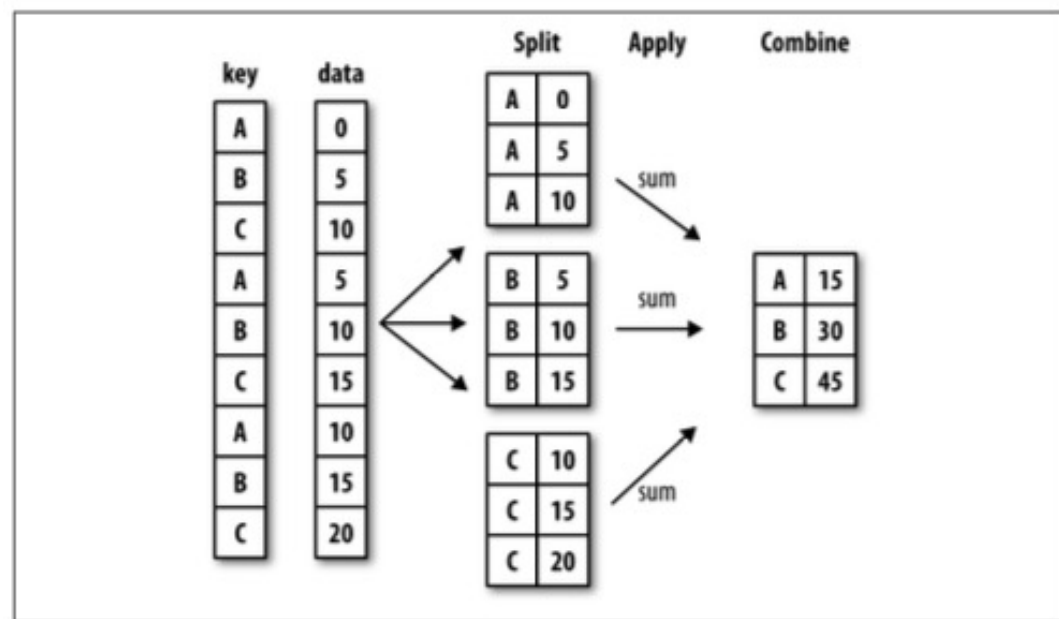


Figure 10-2. Illustration of a group aggregation

Returning to the tipping dataset from before, suppose you wanted to select the top five `tip_pct` values by group. First, write a function that selects the rows with the largest values in a particular column:

```
In [74]: def top(df, n=5, column='tip_pct'):
.....:     return df.sort_values(by=column)[-n:]
```

```
In [75]: top(tips, n=6)
```

```
Out[75]:
```

	total_bill	tip	smoker	day	time	size	tip_pct
109	14.31	4.00	Yes	Sat	Dinner	2	0.279525
183	23.17	6.50	Yes	Sun	Dinner	4	0.280535
232	11.61	3.39	No	Sat	Dinner	2	0.291990
67	3.07	1.00	Yes	Sat	Dinner	1	0.325733
178	9.60	4.00	Yes	Sun	Dinner	2	0.416667
172	7.25	5.15	Yes	Sun	Dinner	2	0.710345



Now, if we group by `smoker`, say, and call `apply` with this function, we get the following:

```
In [76]: tips.groupby('smoker').apply(top)
```

```
Out[76]:
```

		total_bill	tip	smoker	day	time	size	tip_pct
smoker								
No	88	24.71	5.85	No	Thur	Lunch	2	0.236746
	185	20.69	5.00	No	Sun	Dinner	5	0.241663
	51	10.29	2.60	No	Sun	Dinner	2	0.252672
	149	7.51	2.00	No	Thur	Lunch	2	0.266312
	232	11.61	3.39	No	Sat	Dinner	2	0.291990
Yes	109	14.31	4.00	Yes	Sat	Dinner	2	0.279525
	183	23.17	6.50	Yes	Sun	Dinner	4	0.280535
	67	3.07	1.00	Yes	Sat	Dinner	1	0.325733
	178	9.60	4.00	Yes	Sun	Dinner	2	0.416667
	172	7.25	5.15	Yes	Sun	Dinner	2	0.710345

What has happened here? The `top` function is called on each row group from the DataFrame, and then the results are glued together using `pandas.concat`, labeling the pieces with the group names. The result therefore has a hierarchical index whose inner level contains index values from the original DataFrame.

If you pass a function to `apply` that takes other arguments or keywords, you can pass these after the function:

```
In [77]: tips.groupby(['smoker', 'day']).apply(top, n=1, column='total_bill')
```

```
Out[77]:
```

			total_bill	tip	smoker	day	time	size	tip_pct
smoker day									
No	Fri	94	22.75	3.25	No	Fri	Dinner	2	0.142857
	Sat	212	48.33	9.00	No	Sat	Dinner	4	0.186220
	Sun	156	48.17	5.00	No	Sun	Dinner	6	0.103799
	Thur	142	41.19	5.00	No	Thur	Lunch	5	0.121389
Yes	Fri	95	40.17	4.73	Yes	Fri	Dinner	4	0.117750
	Sat	170	50.81	10.00	Yes	Sat	Dinner	3	0.196812
	Sun	182	45.35	3.50	Yes	Sun	Dinner	3	0.077178
	Thur	197	43.11	5.00	Yes	Thur	Lunch	4	0.115982



Beyond these basic usage mechanics, getting the most out of `apply` may require some creativity. What occurs inside the function passed is up to you; it only needs to return a pandas object or a scalar value. The rest of this chapter will mainly consist of examples showing you how to solve various problems using `groupby`.



Apply

Apply: General Split->Apply->Combine

You may recall that I earlier called `describe` on a `GroupBy` object:

```
In [78]: result = tips.groupby('smoker')['tip_pct'].describe()
```

```
In [79]: result
```

```
Out[79]:
```

	count	mean	std	min	25%	50%	75%	\
smoker								
No	151.0	0.159328	0.039910	0.056797	0.136906	0.155625	0.185014	
Yes	93.0	0.163196	0.085119	0.035638	0.106771	0.153846	0.195059	

	max
smoker	
No	0.291990
Yes	0.710345

```
In [80]: result.unstack('smoker')
```

```
Out[80]:
```

	smoker	
count	No	151.000000
	Yes	93.000000
mean	No	0.159328
	Yes	0.163196
std	No	0.039910
	Yes	0.085119
min	No	0.056797
	Yes	0.035638
25%	No	0.136906
	Yes	0.106771
50%	No	0.155625
	Yes	0.153846
75%	No	0.185014
	Yes	0.195059
max	No	0.291990
	Yes	0.710345

```
dtype: float64
```

Inside `GroupBy`, when you invoke a method like `describe`, it is actually just a short-cut for:

```
f = lambda x: x.describe()
grouped.apply(f)
```



Apply > Suppressing the Group Keys

Apply: General Split->Apply->Combine

Suppressing the Group Keys

In the preceding examples, you see that the resulting object has a hierarchical index formed from the group keys along with the indexes of each piece of the original object. You can disable this by passing `group_keys=False` to `groupby`:

```
In [81]: tips.groupby('smoker', group_keys=False).apply(top)
```

```
Out[81]:
```

	total_bill	tip	smoker	day	time	size	tip_pct
88	24.71	5.85	No	Thur	Lunch	2	0.236746
185	20.69	5.00	No	Sun	Dinner	5	0.241663
51	10.29	2.60	No	Sun	Dinner	2	0.252672
149	7.51	2.00	No	Thur	Lunch	2	0.266312
232	11.61	3.39	No	Sat	Dinner	2	0.291990
109	14.31	4.00	Yes	Sat	Dinner	2	0.279525
183	23.17	6.50	Yes	Sun	Dinner	4	0.280535
67	3.07	1.00	Yes	Sat	Dinner	1	0.325733
178	9.60	4.00	Yes	Sun	Dinner	2	0.416667
172	7.25	5.15	Yes	Sun	Dinner	2	0.710345



Apply > Quantile and Bucket Analysis

Apply: General Split->Apply->Combine

Quantile and Bucket Analysis

As you may recall from [Chapter 8](#), pandas has some tools, in particular `cut` and `qcut`, for slicing data up into buckets with bins of your choosing or by sample quantiles. Combining these functions with `groupby` makes it convenient to perform bucket or quantile analysis on a dataset. Consider a simple random dataset and an equal-length bucket categorization using `cut`:

```
In [82]: frame = pd.DataFrame({'data1': np.random.randn(1000),
.....:                        'data2': np.random.randn(1000)})

In [83]: quartiles = pd.cut(frame.data1, 4)

In [84]: quartiles[:10]
Out[84]:
0    (-1.23, 0.489]
1    (-2.956, -1.23]
2    (-1.23, 0.489]
3    (0.489, 2.208]
4    (-1.23, 0.489]
5    (0.489, 2.208]
6    (-1.23, 0.489]
7    (-1.23, 0.489]
8    (0.489, 2.208]
9    (0.489, 2.208]
Name: data1, dtype: category
Categories (4, interval[float64]): [(-2.956, -1.23] < (-1.23, 0.489] < (0.489, 2.208] < (2.208, 3.928]]
```

The Categorical object returned by `cut` can be passed directly to `groupby`. So we could compute a set of statistics for the `data2` column like so:

```
In [85]: def get_stats(group):
.....:     return {'min': group.min(), 'max': group.max(),
.....:             'count': group.count(), 'mean': group.mean()}

In [86]: grouped = frame.data2.groupby(quartiles)

In [87]: grouped.apply(get_stats).unstack()
Out[87]:
```

	count	max	mean	min
data1				
(-2.956, -1.23]	95.0	1.670835	-0.039521	-3.399312
(-1.23, 0.489]	598.0	3.260383	-0.002051	-2.989741
(0.489, 2.208]	297.0	2.954439	0.081822	-3.745356
(2.208, 3.928]	10.0	1.765640	0.024750	-1.929776

These were equal-length buckets; to compute equal-size buckets based on sample quantiles, use `qcut`. I'll pass `labels=False` to just get quantile numbers:

```
# Return quantile numbers
In [88]: grouping = pd.qcut(frame.data1, 10, labels=False)

In [89]: grouped = frame.data2.groupby(grouping)

In [90]: grouped.apply(get_stats).unstack()
Out[90]:
```

	count	max	mean	min
data1				
0	100.0	1.670835	-0.049902	-3.399312
1	100.0	2.628441	0.030989	-1.950098
2	100.0	2.527939	-0.067179	-2.925113
3	100.0	3.260383	0.065713	-2.315555
4	100.0	2.074345	-0.111653	-2.047939
5	100.0	2.184810	0.052130	-2.989741
6	100.0	2.458842	-0.021489	-2.223506
7	100.0	2.954439	-0.026459	-3.056990
8	100.0	2.735527	0.103406	-3.745356
9	100.0	2.377020	0.220122	-2.064111

We will take a closer look at pandas's Categorical type in [Chapter 12](#).



Apply > Example: Filling Missing Values with Group-Specific Values

Apply: General Split->Apply->Combine

Example: Filling Missing Values with Group-Specific Values

When cleaning up missing data, in some cases you will replace data observations using `dropna`, but in others you may want to impute (fill in) the null (NA) values using a fixed value or some value derived from the data. `fillna` is the right tool to use; for example, here I fill in NA values with the mean:

```
In [91]: s = pd.Series(np.random.randn(6))
```

```
In [92]: s[::2] = np.nan
```

```
In [93]: s
```

```
Out[93]:
0      NaN
1   -0.125921
2      NaN
3   -0.884475
4      NaN
5    0.227290
dtype: float64
```

```
In [94]: s.fillna(s.mean())
```

```
Out[94]:
0   -0.261035
1   -0.125921
2   -0.261035
3   -0.884475
4   -0.261035
5    0.227290
dtype: float64
```

Suppose you need the fill value to vary by group. One way to do this is to group the data and use `apply` with a function that calls `fillna` on each data chunk. Here is some sample data on US states divided into eastern and western regions:

```
In [95]: states = ['Ohio', 'New York', 'Vermont', 'Florida',
....:              'Oregon', 'Nevada', 'California', 'Idaho']
```

```
In [96]: group_key = ['East'] * 4 + ['West'] * 4
```

```
In [97]: data = pd.Series(np.random.randn(8), index=states)
```

```
In [98]: data
```

```
Out[98]:
Ohio      0.922264
New York  -2.153545
Vermont   -0.365757
Florida   -0.375842
Oregon     0.329939
Nevada     0.981994
California 1.105913
Idaho     -1.613716
dtype: float64
```



Apply > Example: Filling Missing Values with Group-Specific Values

Apply: General Split->Apply->Combine

Note that the syntax `['East'] * 4` produces a list containing four copies of the elements in `['East']`. Adding lists together concatenates them.

Let's set some values in the data to be missing:

```
In [99]: data[['Vermont', 'Nevada', 'Idaho']] = np.nan
```

```
In [100]: data
Out[100]:
Ohio      0.922264
New York  -2.153545
Vermont    NaN
Florida   -0.375842
Oregon    0.329939
Nevada     NaN
California 1.105913
Idaho      NaN
dtype: float64
```

```
In [101]: data.groupby(group_key).mean()
Out[101]:
East  -0.535707
West   0.717926
dtype: float64
```

We can fill the NA values using the group means like so:

```
In [102]: fill_mean = lambda g: g.fillna(g.mean())
```

```
In [103]: data.groupby(group_key).apply(fill_mean)
Out[103]:
Ohio      0.922264
New York  -2.153545
Vermont   -0.535707
Florida   -0.375842
Oregon    0.329939
Nevada    0.717926
California 1.105913
Idaho     0.717926
dtype: float64
```

In another case, you might have predefined fill values in your code that vary by group. Since the groups have a name attribute set internally, we can use that:

```
In [104]: fill_values = {'East': 0.5, 'West': -1}
```

```
In [105]: fill_func = lambda g: g.fillna(fill_values[g.name])
```

```
In [106]: data.groupby(group_key).apply(fill_func)
Out[106]:
Ohio      0.922264
New York  -2.153545
Vermont    0.500000
Florida   -0.375842
Oregon    0.329939
Nevada   -1.000000
California 1.105913
Idaho   -1.000000
dtype: float64
```



Apply > Example: Random Sampling and Permutation

Apply: General Split->Apply->Combine

Example: Random Sampling and Permutation

Suppose you wanted to draw a random sample (with or without replacement) from a large dataset for Monte Carlo simulation purposes or some other application. There are a number of ways to perform the “draws”; here we use the `sample` method for Series.

To demonstrate, here's a way to construct a deck of English-style playing cards:

```
# Hearts, Spades, Clubs, Diamonds
suits = ['H', 'S', 'C', 'D']
card_val = (list(range(1, 11)) + [10] * 3) * 4
base_names = ['A'] + list(range(2, 11)) + ['J', 'K', 'Q']
cards = []
for suit in ['H', 'S', 'C', 'D']:
    cards.extend(str(num) + suit for num in base_names)

deck = pd.Series(card_val, index=cards)
```

So now we have a Series of length 52 whose index contains card names and values are the ones used in Blackjack and other games (to keep things simple, I just let the ace 'A' be 1):

```
In [108]: deck[:13]
Out[108]:
AH      1
2H      2
3H      3
4H      4
5H      5
6H      6
7H      7
8H      8
9H      9
10H     10
JH      10
KH      10
QH      10
dtype: int64
```

Now, based on what I said before, drawing a hand of five cards from the deck could be written as:

```
In [109]: def draw(deck, n=5):
.....:     return deck.sample(n)

In [110]: draw(deck)
Out[110]:
AD      1
8C      8
5H      5
KC      10
2C      2
dtype: int64
```



Apply > Example: Random Sampling and Permutation

Apply: General Split->Apply->Combine

Suppose you wanted two random cards from each suit. Because the suit is the last character of each card name, we can group based on this and use apply:

```
In [111]: get_suit = lambda card: card[-1] # last letter is suit
```

```
In [112]: deck.groupby(get_suit).apply(draw, n=2)
```

```
Out[112]:
```

```
C  2C    2
```

```
   3C    3
```

```
D  KD   10
```

```
   8D    8
```

```
H  KH   10
```

```
   3H    3
```

```
S  2S    2
```

```
   4S    4
```

```
dtype: int64
```

Alternatively, we could write:

```
In [113]: deck.groupby(get_suit, group_keys=False).apply(draw, n=2)
```

```
Out[113]:
```

```
KC    10
```

```
JC    10
```

```
AD     1
```

```
5D     5
```

```
5H     5
```

```
6H     6
```

```
7S     7
```

```
KS    10
```

```
dtype: int64
```




Apply > Example: Group Weighted Average and Correlation

Apply: General Split->Apply->Combine

Example: Group Weighted Average and Correlation

Under the split-apply-combine paradigm of groupby, operations between columns in a DataFrame or two Series, such as a group weighted average, are possible. As an example, take this dataset containing group keys, values, and some weights:

```
In [114]: df = pd.DataFrame({'category': ['a', 'a', 'a', 'a',  
.....:                                'b', 'b', 'b', 'b'],  
.....:                      'data': np.random.randn(8),  
.....:                      'weights': np.random.rand(8)})
```

```
In [115]: df  
Out[115]:
```

	category	data	weights
0	a	1.561587	0.957515
1	a	1.219984	0.347267
2	a	-0.482239	0.581362
3	a	0.315667	0.217091
4	b	-0.047852	0.894406
5	b	-0.454145	0.918564
6	b	-0.556774	0.277825
7	b	0.253321	0.955905

The group weighted average by category would then be:

```
In [116]: grouped = df.groupby('category')
```

```
In [117]: get_wavg = lambda g: np.average(g['data'], weights=g['weights'])
```

```
In [118]: grouped.apply(get_wavg)
```

```
Out[118]:  
category  
a      0.811643  
b     -0.122262  
dtype: float64
```

As another example, consider a financial dataset originally obtained from Yahoo! Finance containing end-of-day prices for a few stocks and the S&P 500 index (the SPX symbol):

```
In [119]: close_px = pd.read_csv('examples/stock_px_2.csv', parse_dates=True,  
.....:                          index_col=0)
```

```
In [120]: close_px.info()  
<class 'pandas.core.frame.DataFrame'>  
DatetimeIndex: 2214 entries, 2003-01-02 to 2011-10-14  
Data columns (total 4 columns):  
AAPL      2214 non-null float64  
MSFT      2214 non-null float64  
XOM        2214 non-null float64  
SPX        2214 non-null float64  
dtypes: float64(4)  
memory usage: 86.5 KB
```

```
In [121]: close_px[-4:]
```

```
Out[121]:
```

	AAPL	MSFT	XOM	SPX
2011-10-11	400.29	27.00	76.27	1195.54
2011-10-12	402.19	26.96	77.16	1207.25
2011-10-13	408.43	27.18	76.37	1203.66
2011-10-14	422.00	27.27	78.11	1224.58



Apply > Example: Group Weighted Average and Correlation

Apply: General Split->Apply->Combine

One task of interest might be to compute a DataFrame consisting of the yearly correlations of daily returns (computed from percent changes) with SPX. As one way to do this, we first create a function that computes the pairwise correlation of each column with the 'SPX' column:

```
In [122]: spx_corr = lambda x: x.corrwith(x['SPX'])
```

Next, we compute percent change on close_px using pct_change:

```
In [123]: rets = close_px.pct_change().dropna()
```

Lastly, we group these percent changes by year, which can be extracted from each row label with a one-line function that returns the year attribute of each datetime label:

```
In [124]: get_year = lambda x: x.year
```

```
In [125]: by_year = rets.groupby(get_year)
```

```
In [126]: by_year.apply(spx_corr)
```

```
Out[126]:
```

	AAPL	MSFT	XOM	SPX
2003	0.541124	0.745174	0.661265	1.0
2004	0.374283	0.588531	0.557742	1.0
2005	0.467540	0.562374	0.631010	1.0
2006	0.428267	0.406126	0.518514	1.0
2007	0.508118	0.658770	0.786264	1.0
2008	0.681434	0.804626	0.828303	1.0
2009	0.707103	0.654902	0.797921	1.0
2010	0.710105	0.730118	0.839057	1.0
2011	0.691931	0.800996	0.859975	1.0

You could also compute inter-column correlations. Here we compute the annual correlation between Apple and Microsoft:

```
In [127]: by_year.apply(lambda g: g['AAPL'].corr(g['MSFT']))
Out[127]:
```

2003	0.480868
2004	0.259024
2005	0.300093
2006	0.161735
2007	0.417738
2008	0.611901
2009	0.432738
2010	0.571946
2011	0.581987

dtype: float64

Apply > Example: Group-Wise Linear Regression

Apply: General Split->Apply->Combine

Example: Group-Wise Linear Regression

In the same theme as the previous example, you can use `groupby` to perform more complex group-wise statistical analysis, as long as the function returns a pandas object or scalar value. For example, I can define the following `regress` function (using the `statsmodels` econometrics library), which executes an ordinary least squares (OLS) regression on each chunk of data:

```
import statsmodels.api as sm
def regress(data, yvar, xvars):
    Y = data[yvar]
    X = data[xvars]
    X['intercept'] = 1.
    result = sm.OLS(Y, X).fit()
    return result.params
```

Now, to run a yearly linear regression of AAPL on SPX returns, execute:

```
In [129]: by_year.apply(regress, 'AAPL', ['SPX'])
Out[129]:
```

	SPX	intercept
2003	1.195406	0.000710
2004	1.363463	0.004201
2005	1.766415	0.003246
2006	1.645496	0.000000
2007	1.198761	0.003438
2008	0.968016	-0.001110
2009	0.879103	0.002954
2010	1.052608	0.001261
2011	0.806605	0.001514



10.4 Pivot Tables and Cross-Tabulation

A *pivot table* is a data summarization tool frequently found in spreadsheet programs and other data analysis software. It aggregates a table of data by one or more keys, arranging the data in a rectangle with some of the group keys along the rows and some along the columns. Pivot tables in Python with pandas are made possible through the `groupby` facility described in this chapter combined with reshape operations utilizing hierarchical indexing. `DataFrame` has a `pivot_table` method, and there is also a top-level `pandas.pivot_table` function. In addition to providing a convenience interface to `groupby`, `pivot_table` can add partial totals, also known as *margins*.

Returning to the tipping dataset, suppose you wanted to compute a table of group means (the default `pivot_table` aggregation type) arranged by day and smoker on the rows:

```
In [130]: tips.pivot_table(index=['day', 'smoker'])
Out[130]:
```

	day	smoker	size	tip	tip_pct	total_bill
Fri		No	2.250000	2.812500	0.151650	18.420000
		Yes	2.066667	2.714000	0.174783	16.813333
Sat		No	2.555556	3.102889	0.158048	19.661778
		Yes	2.476190	2.875476	0.147906	21.276667
Sun		No	2.929825	3.167895	0.160113	20.506667
		Yes	2.578947	3.516842	0.187250	24.120000
Thur		No	2.488889	2.673778	0.160298	17.113111
		Yes	2.352941	3.030000	0.163863	19.190588

This could have been produced with `groupby` directly. Now, suppose we want to aggregate only `tip_pct` and `size`, and additionally group by `time`. I'll put `smoker` in the table columns and `day` in the rows:

```
In [131]: tips.pivot_table(['tip_pct', 'size'], index=['time', 'day'],
.....:                      columns='smoker')
Out[131]:
```

time	day	size		tip_pct	
		No	Yes	No	Yes
Dinner	Fri	2.000000	2.222222	0.139622	0.165347
	Sat	2.555556	2.476190	0.158048	0.147906
	Sun	2.929825	2.578947	0.160113	0.187250
	Thur	2.000000	NaN	0.159744	NaN
Lunch	Fri	3.000000	1.833333	0.187735	0.188937
	Thur	2.500000	2.352941	0.160311	0.163863



피벗테이블 및 교차표

Pivot Tables and Cross-Tabulation

We could augment this table to include partial totals by passing `margins=True`. This has the effect of adding All row and column labels, with corresponding values being the group statistics for all the data within a single tier:

```
In [132]: tips.pivot_table(['tip_pct', 'size'], index=['time', 'day'],
.....:                    columns='smoker', margins=True)
Out[132]:
```

		size		tip_pct			
		No	Yes	All	No	Yes	All
Dinner	Fri	2.000000	2.222222	2.166667	0.139622	0.165347	0.158916
	Sat	2.555556	2.476190	2.517241	0.158048	0.147906	0.153152
	Sun	2.929825	2.578947	2.842105	0.160113	0.187250	0.166897
	Thur	2.000000	NaN	2.000000	0.159744	NaN	0.159744
Lunch	Fri	3.000000	1.833333	2.000000	0.187735	0.188937	0.188765
	Thur	2.500000	2.352941	2.459016	0.160311	0.163863	0.161301
All		2.668874	2.408602	2.569672	0.159328	0.163196	0.160803

Here, the All values are means without taking into account smoker versus non-smoker (the All columns) or any of the two levels of grouping on the rows (the All row).

To use a different aggregation function, pass it to `aggfunc`. For example, 'count' or `len` will give you a cross-tabulation (count or frequency) of group sizes:

```
In [133]: tips.pivot_table('tip_pct', index=['time', 'smoker'], columns='day',
.....:                    aggfunc=len, margins=True)
Out[133]:
```

		Fri	Sat	Sun	Thur	All
Dinner	No	3.0	45.0	57.0	1.0	106.0
	Yes	9.0	42.0	19.0	NaN	70.0
Lunch	No	1.0	NaN	NaN	44.0	45.0
	Yes	6.0	NaN	NaN	17.0	23.0
All		19.0	87.0	76.0	62.0	244.0

If some combinations are empty (or otherwise NA), you may wish to pass a `fill_value`:

```
In [134]: tips.pivot_table('tip_pct', index=['time', 'size', 'smoker'],
.....:                    columns='day', aggfunc='mean', fill_value=0)
Out[134]:
```

			Fri	Sat	Sun	Thur
Dinner	1	No	0.000000	0.137931	0.000000	0.000000
		Yes	0.000000	0.325733	0.000000	0.000000
	2	No	0.139622	0.162705	0.168859	0.159744
		Yes	0.171297	0.148668	0.207893	0.000000
	3	No	0.000000	0.154661	0.152663	0.000000
		Yes	0.000000	0.144995	0.152660	0.000000
	4	No	0.000000	0.150096	0.148143	0.000000
		Yes	0.117750	0.124515	0.193370	0.000000
	5	No	0.000000	0.000000	0.206928	0.000000
		Yes	0.000000	0.106572	0.065660	0.000000
...			...	...	...	...
Lunch	1	No	0.000000	0.000000	0.000000	0.181728
		Yes	0.223776	0.000000	0.000000	0.000000
	2	No	0.000000	0.000000	0.000000	0.166005
		Yes	0.181969	0.000000	0.000000	0.158843
	3	No	0.187735	0.000000	0.000000	0.084246
		Yes	0.000000	0.000000	0.000000	0.204952
	4	No	0.000000	0.000000	0.000000	0.138919
		Yes	0.000000	0.000000	0.000000	0.155410
	5	No	0.000000	0.000000	0.000000	0.121389
		Yes	0.000000	0.000000	0.000000	0.173706
	6	No	0.000000	0.000000	0.000000	0.173706
		Yes	0.000000	0.000000	0.000000	0.173706

[21 rows x 4 columns]



See **Table 10-2** for a summary of `pivot_table` methods.

Table 10-2. `pivot_table` options

Function name	Description
<code>values</code>	Column name or names to aggregate; by default aggregates all numeric columns
<code>index</code>	Column names or other group keys to group on the rows of the resulting pivot table
<code>columns</code>	Column names or other group keys to group on the columns of the resulting pivot table
<code>aggfunc</code>	Aggregation function or list of functions (' mean ' by default); can be any function valid in a groupby context
<code>fill_value</code>	Replace missing values in result table
<code>dropna</code>	If <code>True</code> , do not include columns whose entries are all <code>NA</code>
<code>margins</code>	Add row/column subtotals and grand total (<code>False</code> by default)



피벗테이블 및 교차표 > Cross-Tabulations: Crosstab

Pivot Tables and Cross-Tabulation

Cross-Tabulations: Crosstab

A cross-tabulation (or *crosstab* for short) is a special case of a pivot table that computes group frequencies. Here is an example:

```
In [138]: data
Out[138]:
```

	Sample	Nationality	Handedness
0	1	USA	Right-handed
1	2	Japan	Left-handed
2	3	USA	Right-handed
3	4	Japan	Right-handed
4	5	Japan	Left-handed
5	6	Japan	Right-handed
6	7	USA	Right-handed
7	8	USA	Left-handed
8	9	Japan	Right-handed
9	10	USA	Right-handed

As part of some survey analysis, we might want to summarize this data by nationality and handedness. You could use `pivot_table` to do this, but the `pandas.crosstab` function can be more convenient:

```
In [139]: pd.crosstab([data.Nationality, data.Handedness], margins=True)
Out[139]:
```

Handedness	Left-handed	Right-handed	All
Nationality			
Japan	2	3	5
USA	1	4	5
All	3	7	10

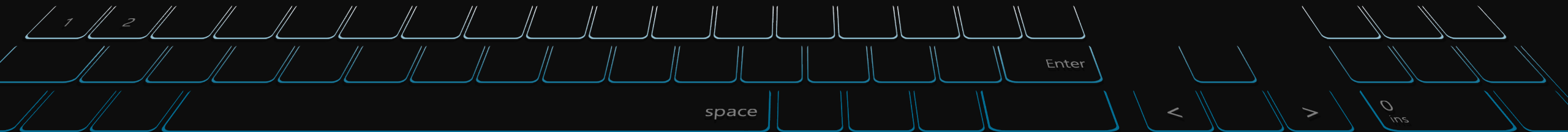
The first two arguments to `crosstab` can each either be an array or Series or a list of arrays. As in the tips data:

```
In [140]: pd.crosstab([tips.time, tips.day], tips.smoker, margins=True)
Out[140]:
```

time	day	No	Yes	All
Dinner	Fri	3	9	12
	Sat	45	42	87
	Sun	57	19	76
	Thur	1	0	1
Lunch	Fri	1	6	7
	Thur	44	17	61
All		151	93	244

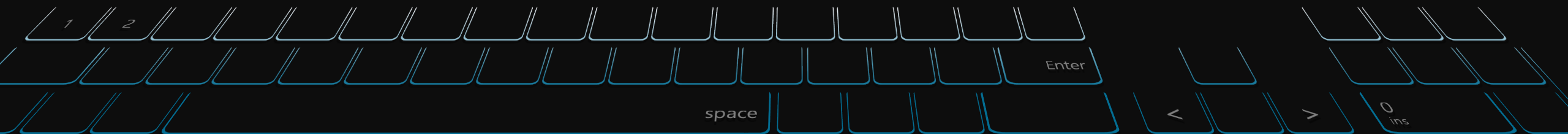
Q&A and Break Time

질의응답 및 휴식 시간 (5분)



2부: 데이터 집계와 그룹 연산

- 1) 데이터 분석가의 역할
- 2) 기업들의 AI 이용 사례
- 3) AI 산업 내 직군 상세 분류
- 4) 진로 탐색 및 취업 준비 노하우
- 5) 프로젝트 팀 구성 및 기획서 작성 기한 및 요령





데이터 분석가의 역할

Roles of Data Analyst

1. 서비스/제품을 이용하는 모바일 유저들의 행태 Data를 정제/가공/적재하며, 대시보드와 리포트 생성
2. 스쿼드에서 Data-driven한 의사결정을 할 수 있도록 정량적 수치와 분석 결과를 제공
3. 목표를 달성하기 위한 지표와 가설을 설정하고, 이를 검증할 수 있는 다양한 A/B 테스트를 설계 및 실험 진행
4. 실험을 통해 얻은 결과를 정리해두는 것에서 그치지 않고 다양한 데이터 분석 기법(Cohort, Funnel 분석 등)을 활용해 인사이트를 도출하고 서비스에 적용점을 찾음
5. 끊임없는 이터레이션을 통해 이슈를 해결하고 서비스를 고도화하여 유저들에게 최고의 경험을 선사
6. Product, Marketing, Corporate Development, Finance 등 다양한 팀과의 협업을 통해 비즈니스 문제를 정의하고, 이를 해결하기 위한 가설을 수립
7. 기본적으로는 소속 스쿼드의 서비스/제품 분석을 하지만, 더 나아가 전사 차원의 관점에서 제품의 효율성, 비용 및 비즈니스 프로세스 등을 평가하여 제품 간 Cross Activation 영향도, 개선/성장 방향성, 프로젝트 우선순위 등에 대한 솔루션을 제공
8. Product Owner와 함께 서비스 및 전사의 방향을 결정하는 중요한 전략가로서의 역할도 수행 가능



데이터 분석가 채용시 우대 기능

Roles of Data Analyst

1. SQL을 사용해 직접 데이터를 추출하고 정제해보신 분
2. 모바일 서비스 데이터 분석 방법(Customer Lifetime Value, Retention, Cohort Analysis 등)에 대한 높은 수준의 이해도를 가진 분
3. 지표의 성장을 위해 다양한 가설을 설정해보고 실험하여 직접 서비스를 개선해보신 분
4. Data Warehouse 구축 지식이나 경험 보유하신 분
5. 데이터 시각화(Tableau, Superset 등)에 대한 경험 있으신 분



데이터 분석가 채용시 이력서에 담으면 좋은 내용

Roles of Data Analyst

1. 그동안 해오신 업무 중 임팩트 있었던 프로젝트를 구체적으로 적어주세요.
2. 진행했던 프로젝트의 문제정의-가설설정-실험설계 및 검증-결과의 과정이 드러나면 좋아요.
3. 데이터 분석으로 서비스 이용 유저에 대해 깊이 이해하고 이를 통해 액션을 제시하신 경험이 있다면 적어주세요.
4. 모바일 서비스 데이터 분석 방법(LTV, AARRR, Cohort, Funnel 등)을 깊게 활용해봤는지 확인하고 있어요.
5. 목표를 달성했다면 이탈률, 전환율, 매출액 등 지표의 개선을 수치로 나타내주면 좋아요.
(외부 공개가 민감한 사항일 경우, 해당 부분은 제외하여 주세요.)



데이터 분석가의 역할

Roles of Data Analyst

Toss Career 내 Data Analyst 채용공고

https://toss.im/career/job-detail?job_id=4103438003

https://toss.im/career/job-detail?job_id=4154060003

https://toss.im/career/job-detail?job_id=4169147003

https://toss.im/career/job-detail?job_id=4071102003

https://toss.im/career/job-detail?job_id=4571130003



Data Analyst

토스뱅크

Data Analyst

토스증권

Data Analyst

토스페이먼츠

Data Analyst

토스

Data Analyst (Global)

토스



데이터 분석가의 역할

Roles of Data Analyst

Kakao, Data Analyst 채용공고

<https://careers.kakao.com/jobs?skillSet=&part=TECHNOLOGY&company=KAKAO&keyword=data%20analyst&employeeType=&page=1>

https://kakaopaysec.ninehire.site/job_posting/UpBwTty2

<https://www.wanted.co.kr/wd/46202>

<https://www.catch.co.kr/NCS/RecruitInfoDetails/269223>

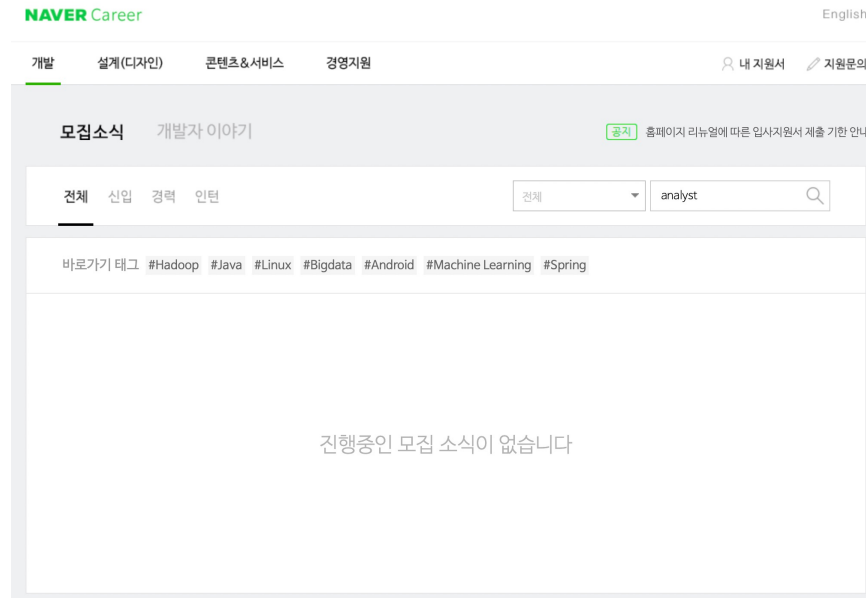
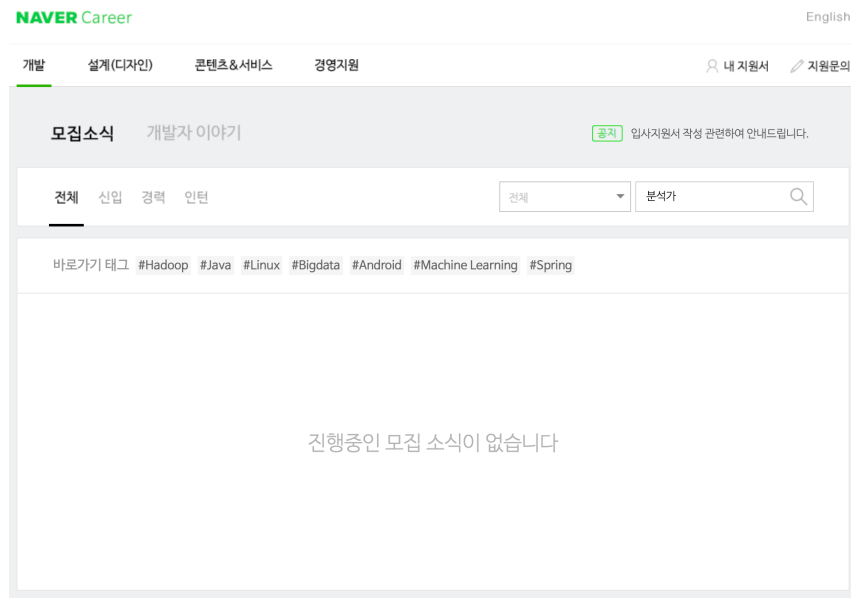


데이터 분석가의 역할

Roles of Data Analyst

Naver Recruit 홈페이지 내 Data Analyst 채용공고

<https://recruit.navercorp.com/rcrt/list.do?sw=data+analyst>





데이터 분석가의 역할

Roles of Data Analyst

네카라쿠배당토직야 내 Data Analyst 채용공고

https://team.daangn.com/jobs/?q=data+analyst#_filter



회사 소개

팀 문화

채용공고

자주 묻는 질문

Data (3)

전체 고용형태

검색

데이터 분석가 (Junior Data Analyst)

당근마켓

정규직

데이터 분석가 (Senior Data Analyst)

당근마켓

정규직

데이터 사이언티스트 디시전 (Data Scientist, Decision)

당근마켓

정규직

네카라쿠배/당토직야/몰두센/마에그원

네카라쿠배

NAVERkakaoLINEcoupang배달의민족

목차

1. 개요

2. 상세

3. 확장판

4. 둘러보기

1. 개요

[편집]

대한민국 IT업계에서 네이버, 카카오, 라인, 쿠팡, 배달의민족(우아한형제들)을 함께 묶어부르는 말이다. 다르게 표현하자면 대한민국 IT기업 5대 공통.

미국의 빅테크 기업을 일컫는 MAGA, FAANG, Magnificent 7과 같이 한국의 대형 IT 서비스 기업을 모아서 부르는 말이다. 다만, MAGA 등이 주식쪽 용어라면 이쪽은 개발직군을 노리는 취준생들 위주로 쓰이는 것이 강하다. 미국 대형 테크 기업과 비교하면 작은 규모이지만, 네이버, 카카오는 각각 2021년 상반기 코스피 시가총액 3위를 기록했으며(우선주 제외), 라인은 한일합작 기업임에도 불구하고 일본 메신저 시장을 독점하고 있다. 쿠팡은 한국 유니콘 기업 최초로 NYSE에 상장되었으며 상장 첫날 100조가 넘는 시가 총액을 기록했다. 배달의민족은 5조에 가까운 금액으로 독일 기업인 딜리버리히어로(Delivery Hero SE)와 M&A를 체결하였다.

2. 상세^[1]

[편집]

기업명	설립일	시가총액(원)	매출(원)	업종
네이버	1999.06.02.	34.6조	8.2조	검색 포털
카카오	1995.02.16. ^[2]	25조	7.1조	메신저
라인	2011.06.23.	27.1조 ^[3]	15.6조	메신저, 검색 포털
쿠팡	2010.07.01.	42조	37.9조	이커머스
배달의민족 ^[4]	2011.03.10.	7.6조 ^[5]	2.9조	배달 플랫폼

네카라쿠배/당토직야/몰두센/마에그원

네카라쿠배당토직야?



이 단어의 기원은 유명 커뮤니티인 디시인사이드의 프로그래밍 갤러리에서 나온 유행어이다. 개발자 취준생들이 취업을 준비할 때 가고 싶어하는 인기있는 회사들을 묶어부르다가 굳어진 말이다. 처음에는 제일 유명한 '네이버, 카카오, 라인'을 줄인 '네카라'라고만 부르기 시작하다가 점차 그 단어에 여러 IT 후발주자인 회사들이 붙으면서 '네카라 > 네카라쿠배 > 네카라쿠배당토 > 네카라쿠배당토직야'로 확장되기 시작했다.

이러한 단어가 점차 익명 직장인 커뮤니티 플랫폼 '블라인드'를 중심으로 사용빈도가 높아지면서, 개발자 취준생들에겐 고3의 '서연고 서성한 중경외시'처럼 모르는 사람이 없을 정도이다.

네카라쿠배/당토직야/몰두센/마에그원

갯기업 '몰두센' 전격비교

		두나무	
	AI 머신러닝 광고 마케팅 솔루션	블록체인 증권거래	B2B 온라인 채팅
복지	자율성은 최고인 몰로코 -완전 자율 출퇴근제 -휴가 무제한 -25,000 상당의 점심 제공 -유급 육아휴직 -가족 건강검진 -스톡옵션	아낌없이 주는 두나무 - 점심, 저녁, 회식 식대 제공 -음료, 과자 제공 -고시양 장비 제공 -인당 30만원 업무용 장비 추가 제공 -무이자 대출 -가족까지 400만원 상당의 건강검진	400가량 지원하는 센드버드 -1년부터 스톡옵션 가능 -400만원 상당의 지원금 -채택근무 -각종 지원금 제공 (출퇴근, 외국어)
채용	신입경력 적극채용 -부트캠프 참여 -온라인 채용박람회 -신입 대규모 채용 -간편한 지원	경력 중심 수시채용 -간편한 지원 -대규모 채용도 경력중심 -경력 수시 채용	신입경력 적극채용 -온라인 채용박람회 -신입 대규모 채용 -간편한 지원



Google

물두센 네카라쿠배당토직야

×

이미지

동영상

뉴스

쇼핑

지도

도서

항공편

금융

검색결과 약 1,110개 (0.34초)

캐치

<https://www.catch.co.kr>

News > RecruitNews

네카라 제친 갓기업 물두센 전격비교

네카라 제치고 입사하고 싶은 기업으로 우뚝선 '물두센' · 연봉, 복지 모두 우수한 수준 · 성장하는 기업인만큼 신규 채용도 활발해 · “네카라쿠배당토직야” · '물두센' 도대체 ...

매일경제

<https://www.mk.co.kr>

news

개발자 신의 직장 지각변동... `네카라` 앞에 `물두센`

2022. 4. 19. — ‘네카라쿠배당토직야’ 위에 ‘물두센’. 정보기술(IT) 업계 개발자들 사이에서 ‘신의 직장’ 지형도가 달라지고 있다. 네이버 카카오 라인 쿠팡 배달의 ...

jobjobking.com

<https://jobjobking.com>

...

네카라쿠배 당토직야 물두센 이름 뜻 알아보기 - HR Masterships

2022. 8. 4. — 각각 당근마켓, 토스, 직방, 야놀자를 의미합니다. 1그룹과의 차이점은 최근 개발자 연봉 인상 과정에서 매우 공격적으로 연봉을 올렸다는 부분인데, 현재 ...

Naver

<https://post.naver.com>

viewer > postView

네이버, 카카오, 라인? 이제는 '물두센' - naver Post

2022. 5. 31. — 혹시나 모르는 분들이 있을까봐 알려드리면, '네카라쿠배당토직야'는 취준생, 특히나 개발자들의 선호가 높은 9개 기업(네이버, 카카오, 라인, 쿠팡, ...

네카라쿠배/당토직야/몰두센/마에그원



네카라

네이버랩스
NAVER LABS

네이버랩스는 네이버의 미래 기술을 책임지는 R&D 전문 자회사입니다. 한국과 유럽의 우수한 연구자들이 컴퓨팅, AR 등의 연구에 매진하고 있습니다. 가장 독창적이며 앞선 기술력을 통해 사람, 머신, 공간, 정보를 새롭게 변화하는 것이 우리의 미션입니다. [바로가기](#)



네이버클라우드
NAVER Cloud

네이버클라우드의 디지털 혁신을 이끄는 하이퍼스케일 기술 집합체로, 팀 네이버의 다양한 기술과 서비스 온 B2B 서비스로 제공하고 있습니다. 초대규모 AI 하이퍼클로버X부터 글로벌 협업 도구 '네이버웍스', 그리고 지식의 혁신과 성장을 돕는 데 필요한 모든 기반을 지원하며, 누구나 팀 네이버의 노하우와 하이퍼스케일 기술 집합체를 활용할 수 있도록 정진하고 있습니다. [바로가기](#)



6.2. AI·헬스케어 중심 미래 성장동력

6.2.1. 인공지능/헬스케어/블록체인

- **카카오브레인**: AI 개발 및 연구 기업.
- **카카오엔터프라이즈**: 카카오 i 등 AI 기술 연구 개발 및 이를 활용한 통합플랫폼과 다양한 기업 서비스를 제공한다.
- **카카오헬스케어**: 헬스케어 전문 자회사.
- **그라운드X**: 블록체인 자회사.

6.2.2. 스타트업/생태계

- **카카오벤처스**: 투자전문 자회사. 초기 스타트업 발굴에 초점이 맞춰져 있음
- **카카오인베스트먼트**: 투자전문 자회사. 수익화 목적 투자에 초점이 맞춰짐
 - **스테이지파이브**: 카카오페이 스마트폰샷, 알뜰폰 핀플레이 운영.
 - **야나두**: 야나두, 카카오키즈 서비스 운영.
 - 픽셀허브: 키즈 서비스 제작.
 - **와이어트**: 카카오헤어샵 서비스, 닥터포헤어 샴푸 등 헤어 관련 사업
 - **파이디지털헬스케어**: 의료 IT 전문 기업.
 - **키즈노트**: 키즈노트 서비스 운영.
 - 에이원즈: 장난감 쇼핑을 운영.

라인 인공지능

AI 로 재탄생하는 새로운 나, 라인 'AI 셀카' 출시 - LINE

2023. 6. 2. — 'AI 셀카'는 별도의 앱 다운로드 없이 라인 메신저 앱 내 프로필 화면의 프로필 스튜디오에서 바로 사용 가능하다. 사용자가 자신의 사진을 10장에서 20장 ...

linepluscorp.com
<http://linepluscorp.com/news>

AI로 돌아보는 나의 어린 시절, 라인 'AI 베이비' 출시 - LINE

2023. 8. 17. — AI의 학습 기술을 활용해 사용자와 닮은 아이의 이미지를 생성해 주어, 실제 어린 시절 사진과 비교해 보는 재미를 제공한다. 또, 여아와 남아 이미지가 ...

LINE
<https://help.line.me/line/android/categoryId>

AI 셀카/AI 사진관

이미지 생성 방법 1) 홈 > 상단의 내 프로필 아이콘이나 이름 선택 2) 스튜디오 > AI 셀카/AI 사진관/AI 베이비 중 원하는 서비스 선택 3) 하단에 표시된 버튼 선택

인공지능신문
<https://www.aitimes.kr/news/articleView>

인공지능으로 돌아보는 나의 어린 시절은?...라인 'AI 베이비' ...

2023. 8. 17. — AI 베이비는 라인 메신저 내 프로필 화면의 스튜디오에서 바로 사용 가능하고, 이미지 생성 소요 시간에 따라 30장의 이미지를 24시간 후에 생성: 160 ...

당근마켓 인공지능

×

이미지

동영상

뉴스

쇼핑

도서

지도

항공편

금융

검색결과 약 5,650,000개 (0.36초)

Medium

https://medium.com › daangn › 당근마켓-ai의-데이터...

당근마켓 AI의 데이터 활용 방법

2021. 5. 21. — 당근마켓 AI가 궁극적으로 해야하는 일은 이해 학습(Representation learning)이라고 생각해요. 게시글을 잘 이해 학습할 수 있다면 자연스럽게 게시글을 ...

이데일리

https://www.edaily.co.kr › news › read

당근마켓 "AI로 동네상권 살릴 것"[AI코리아대상]

2022. 9. 25. — 당근마켓은 AI 머신러닝 기술을 통해 **개개인에게 맞춤형 콘텐츠를 제공** 중이다. AI를 통해 스펀, 사기, 불법 요소를 수초 이내에 자동으로 체크한다. 이를 ...

오픈애즈

https://www.openads.co.kr › content › contentDetail

'빨리 팔고 싶으면 3000원'...당근마켓의 실험 / "생성형 AI가 ...

2023. 4. 25. — 구글이 광고 사업에 생성형 인공지능(AI) 기술을 도입할 계획인 것으로 알려졌습니다. 파이낸셜 타임스(FT)에 따르면 구글은 광고주들과의 내부 ...

조선일보

https://it.chosun.com › data › html_dir › 2019/10/04

[전문물 시대] ㉮모바일 중고시장 혁신한 당근마켓 "AI로 이웃 간 ...

2019. 10. 6. — 당근마켓은 여기에 인공지능(AI) 기술도 접목했다. 머신러닝에 가끔 데이터를 학습시켜, 가품 게시물에 올라오면 자동으로 인식한다. 판매불가 상품인 ...

토스 인공지능

×

한겨레

https://www.hani.co.kr › economy › economy_general

토스 AI가 '가짜 신분증' 600건 잡았다 : 경제일반

2023. 8. 14. — 토스뱅크가 자체 개발한 인공지능 기반 실시간 신분증 이상 탐지 시스템을 통해 지난 6개월간 600여 건의 가짜 신분증을 잡아냈다고 14일 밝혔다.

토스

https://toss.im › slash-22 › sessions

물 흐르듯 자연스러운 ML 서비스 만들기

회사마다 다른 AI, ML 적용 방식. 토스에서는 어떻게 사용하고 있을까요? 대규모 서비스에 최적화된 기술 스택과 인프라를 기반으로 머신러닝 서비스를 만들고 배포 ...

브런치스토리

https://brunch.co.kr › ...

토스가 겪은 생성AI의 도전과 한계 이야기

인공지능을 적용한 한 예로는, 토스 모바일에 적용된 '스피치 투 텍스트(Speech To Text)' 서비스가 있다. 휴대폰 계약자 변경시 인공지능이 고객 대신 이전 통신사의 ...

지디넷코리아

https://zdnet.co.kr › view

토스뱅크 "가짜 신분증 AI가 잡았다"

2023. 8. 14. — 인터넷전문은행 '토스뱅크'가 인공지능(AI) 기반 실시간 신분증 이상 탐지 시스템을 통해 지난 6개월 간 약 600건의 가짜 신분증을 잡아냈다고 14일 ...

이데일리

https://www.edaily.co.kr › news › read

토스 앱에서 챗GPT 바로 쓴다

2023. 3. 9. — 토스 앱에서 오픈AI의 인공지능(AI) 챗봇 서비스 챗GPT를 바로 쓸 수 있게 됐다. 오픈AI가 외부 서비스에 챗GPT를 결합할 수 있도록 인터페이스(API) ...

직방 인공지능

이미지

뉴스

동영상

쇼핑

도서

지도

항공편

금융

검색결과 약 618,000개 (0.76초)

지디넷코리아

https://zdnet.co.kr › view

직방, 인공지능 빅데이터 솔루션 '직방RED' 출시

2023. 3. 29. — 직방(대표 안성우)이 부동산 전문가를 위한 인공지능(AI) 빅데이터 솔루션 직방RED(Real Estate Data) 서비스를 출시한다고 29일 밝혔다.

연합뉴스

https://www.yna.co.kr › 최신기사

직방, 부동산 전문가 AI 빅데이터 솔루션 '직방RED' 정식 출시

2023. 3. 29. — 직방 레드는 부동산 빅데이터에 데이터 사이언스 기술을 접목해 고품질 정보를 제공하는 전문가용 부동산 통계 데이터 도구다. ADVERTISEMENT. 2019 ...

머니투데이

https://news.mt.co.kr › mtview

B2B 공략하는 직방, 부동산 전문가 위한 AI 빅데이터 솔루션 ...

2023. 3. 29. — 직방RED는 부동산 빅데이터에 데이터 사이언스 기술을 접목해 고품질 정보를 제공하는 전문가용 부동산 통계 데이터 툴이다. 2019년 베타로 시작한 이후 ...

인공지능신문

https://www.aitimes.kr › news › articleView

AI가 부동산 정보 확인한다... KT, 직방에 AI 컨택센터 솔루션 ...

2021. 6. 17. — 사람이 하던 매물 확인 전화를 AI 보이스봇이 대신하고, 체계적 데이터 관리 지원. KT가 종합 프롭테크 기업 직방과 부동산 분야 AI컨택센터 사업협력 ...

야놀자 인공지능

서울경제

https://www.sedaily.com › NewsView

[단독] 야놀자, AI에 1000억 투입...에드테크 '데이블' 인수

2021. 11. 23. — 기업가치 10조 원을 넘어서 '데카콘'으로 성장한 야놀자가 인공지능(AI)과 빅데이터 등 기술 역량 강화에 투자를 확대하고 있다.

소프트웨어야 놀자

https://www.playsw.or.kr › artificial › view › playswtv

인공지능(AI)이란?

사람의 지능을 흉내내는 소프트웨어, 프로그램, 인공지능 장치 모두를 바로 '인공지능'이라고 합니다. 인공지능은 다양한 사람들의 일의 방식과 생활의 변화를 만들어 ...

이슈

https://www.ebssw.kr › edc › cultursensDetailView

이슈 | EBS 소프트웨어

AI야, 놀자. 인공지능의 개념을 알고, 인공지능의 활용으로 달라진 생활 모습을 이야기하고자 한다. 강좌 검색 search. 인공지능이 뭐예요? 01. 인공지능이 뭐예요?

매일경제

https://www.mk.co.kr › news

야놀자·KT의 `AI호텔` 합작...이젠 대기업이 스타트업 생태계 ...

2021. 12. 20. — 'AI(인공지능)호텔' 예약부터 객실서비스 제공, 그리고 객실관리까지 모든걸 컴퓨터(AI)가 하는 호텔을 말하는데요. 이 AI호텔을 광주광역시에 처음 ...

비즈니스포스트

https://www.businesspost.co.kr › ...

[현장] 야놀자 이수진 대담한 목표, 인터파크트리플에 AI 접목 ...

2023. 6. 20. — 이수진 야놀자 총괄대표이사가 지난해 4월 인수한 인터파크의 비전을 발표했다. AI기술을 접목해 국내 패키지 상품 역량을 강화해 2028년 인바운드 5 ...

물두센

물로코 인공지능

지디넷코리아

https://zdnet.co.kr › view

[AI기업] 물로코 "데이터를 돈으로 만들어줘...올해 매출 5천억"

2021. 5. 25. — 2013년 설립된 물로코는 고객(회사)이 가진 데이터를 분석, 돈이 되게 해주는 회사다. 이를 위한 '물'로 인공지능(AI)과 데이터 분석을 사용한다. 배달의 ...

AITA임스

https://www.aitimes.com › news › articleView

물로코, ML으로 효과 극대화 한 온라인 광고 플랫폼 소개

2022. 8. 24. — 인공지능(AI)을 활용해 온라인 광고 효율을 높일 수 있는 자동 광고 플랫폼을 국내에서도 사용할 수 있게 됐다.머신러닝(ML) 솔루션 전문기업 물 ...

Moloco

https://www.moloco.com › ...

물로코

커머스 플랫폼을 위한 물로코. 자체 퍼포먼스 광고 비즈니스를 구축해 새로운 수익원을 창출하는 동시에 셀러의 매출을 높이고 사용자 경험을 향상시킬 수 있습니다. .

roa.ai

https://engine.roa.ai › articles

[물로코] 실리콘 벨리에서 인정받은 머신러닝 애드테크 스타트업

2023. 1. 13. — 물로코 솔루션은 고객사가 보유한 소비자의 앱 이용 행태를 인공지능(AI)으로 분석하여 맞춤형 광고를 제공하는 수익화(Monetization) 모델에 기반을 ...

한국경제

https://www.hankyung.com › article

AI로 적자 극복 돕는다...내년 미상장 도전

2023. 7. 11. — 물로코는 머신러닝 기반 솔루션 기업이다. 빅데이터를 분석해 고객사가 효율적으로 광고를 집행할 수 있도록 지원한다. 소비자와 관련한 정보들을 결합해 ...

두나무 인공지능

비즈워치

http://news.bizwatch.co.kr › mobile › 2023/06/21

"고객편의 향상·투자자 보호"...두나무, AI 도입 확대

2023. 6. 21. — 블록체인·핀테크 기업 두나무가 인공지능(AI)을 도입해 고객 편의를 향상시키고 투자자 보호에 앞장서고 있다. 두나무는 21일 증권플러스에 챗GPT ...

dt.co.kr

http://m.dt.co.kr › contents

두나무, 투자정보 제공 AI `우디` 출시 - 디지털타임스

2023. 6. 21. — 우디는 챗GPT를 기반으로 주가, 뉴스, 공식, 수급현황 등을 학습시킨 인공지능 대화형 서비스다. 빠르게 변하는 증시 상황에 맞춰 투자 정보를 제공하기 ...

한국경제TV

https://www.wowtv.co.kr › NewsCenter › News › Read

" 두나무, AI 서비스 도입 " ...관련주 일제히 강세

2023. 6. 22. — 전일 두나무는 자사의 증권 애플리케이션 증권플러스에서 챗GPT 기반 인공지능(AI) 서비스인 '우디'를 공식 출시했다고 밝혔다. '우디'는 실시간으로 ...

신아일보

https://www.shinailbo.co.kr › news › articleView

두나무, 증권플러스 챗GPT 기반 AI 서비스 '우디' 출시

2023. 6. 21. — 두나무는 지난 20일 국민 증권 애플리케이션(앱) 증권플러스에서 챗GPT 기반 인공지능(AI) 서비스 '우디'를 공식 출시했다고 21일 밝혔다.



센드버드 인공지능

지디넷코리아

https://zdnet.co.kr › view

센드버드, 생성AI 기술 탑재한 '세일즈포스 커넥터' 출시 ...

2023. 8. 24. — 커스터마이징 가능한 대화형 AI 챗봇 탑재, 고객 상담 품질의 비약적 향상 기대. 시연 화면. 국내 최초로 실리콘벨리의 유니콘 기업에 등극한 센드 ...

Sendbird

https://sendbird.com › ...

센드버드 - 채팅과 영상통화 개발의 완벽한 솔루션 | Sendbird

AI와 결합된 채팅, 통화, 라이브 스트리밍 API 및 SDK로 세계 최고의 인앱 대화 환경을 빠르게 구축하고 잔존을 등 성과 지표를 개선하세요.

Sendbird 요금제 · Sendbird 채용 · Products: Chat Messaging · 알림 메시지

벤처스퀘어

https://www.venturesquare.net › ...

센드버드, 노코드 생성형 AI 챗봇 '스마트 어시스턴트' 출시

2023. 7. 13. — 코딩 없이 센드버드의 대시보드에서 손쉽게 스마트 어시스턴트를 배포할 수 있으며, 채팅 UI 키트를 이용해 스마트 어시스턴트가 탑재된 AI 채팅 앱을 단 ...

AITA임스

https://www.aitimes.com › news › articleView

센드버드, 생성 AI 탑재한 '세일즈포스 커넥터' 출시 ...

2023. 8. 24. — 세일즈포스 커넥터는 노코드 생성 인공지능(AI) 챗봇 '스마트 어시스턴트'를 도입하는 등 새 기능들을 추가했다. 센드버드의 커뮤니케이션 플랫폼 안에서 ...

마이리얼트립/에이블리

마이리얼트립 인공지능

이미지 뉴스 동영상 쇼핑 지도 도서 항공편 금융

검색결과 약 620,000개 (0.35초)

지디넷코리아

https://zdnet.co.kr > view

마이리얼트립, 챗GPT 연동 'AI 여행플래너' 출시

2023. 2. 23. — 마이리얼트립이 이번에 선보인 AI 여행플래너 서비스는 챗GPT를 연동, 인공지능과의 대화를 통해 여행 일정을 계획할 수 있는 것은 물론 맛집, 명소, 날씨 ...

위시켓

https://yozm.wishket.com > magazine > detail

48시간만에 국내 여행 업계 최초 AI 서비스 만든 사연 - 요즘IT

2023. 3. 28. — 마이리얼트립이 ChatGPT를 도입해 만든 '여행플래너'는 론칭까지 모두 48시간 밖에 걸리지 않았다고 합니다. 요즘IT가 마이리얼트립을 만나 그 뜨거웠 ...

여행신문

https://www.traveltimes.co.kr > news > articleView

마이리얼트립, 챗GPT 연동한 AI 여행플래너 론칭 ...

2023. 2. 24. — 마이리얼트립 AI 여행플래너 서비스는 인공지능과의 대화를 통해 여행 일정을 계획할 수 있는 것은 물론 맛집, 명소, 날씨, 팁, 여행지 추천 등 여행 관련 ...

히치하이커닷컴

https://hitchhickr.com > myrealtrip_chatgpt

마이리얼트립의 챗GPT 이용한 'AI여행플래너' 써보니?

2023. 2. 16. — 마이리얼트립이 AI 여행플래너라는 신규 서비스를 열었네요. 최근 화제를 모으고 있는 챗GPT를 탑재한 기능입니다. 현재 웹사이트에서는 제공하지 ...

하이테크정보

http://www.hitech.co.kr > news > articleView

글로벌 여행 기업 '챗GPT' 경쟁 치열---마이리얼트립 등 국내 ...

2023. 4. 11. — 마이리얼트립은 국내여행업계 최초로 챗GPT를 연동한 'AI 여행플래너'를 앱에 구축했다. AI 여행플래너는 여행 일정을 계획하게 도와주고 맛집, 명소, ...

에이블리 인공지능

이미지 동영상 뉴스 쇼핑 도서 지도 항공편 금융

검색결과 약 384,000개 (0.39초)

디지털 인사이드

https://ditoday.com > 에이블리-흑자-전환-성공-카테...

에이블리 흑자 전환 성공, 카테고리 확장과 인공지능(AI)이 핵심

2023. 4. 17. — 에이블리 흑자 전환 성공, 카테고리 확장과 인공지능(AI)이 핵심 · 월간 손익분기점(BEP) 달성... · 단발적 실적 아닌 매출 2배 성장 동반된 결과... · 글로벌 ...

에이블리 팀

https://ably.team > news > YnTMRxEAAcAmDw8

에이블리가 고속 성장할 수밖에 없는 이유 “시장과 기술”

2022. 5. 6. — 탄탄한 기술력을 바탕으로, 상품력을 갖춘 셀러와 유저를 연결하는 역할을 하고 있는데요. 에이블리는 업계 최초로 자체 개발한 'AI 개인화 추천' 서비스 ...

연함뉴스

https://www.yna.co.kr > 최신기사

에이블리 "AI 개인화 기술 활용한 '4세대 쇼핑몰' 급성장"

2023. 6. 30. — 스타일 커머스 플랫폼 에이블리는 지난달 인공지능(AI) 개인화 추천 기술을 활용한 '4세대 쇼핑몰'의 주문 수가 지난해 같은 달과 비교해 최대 24배 증가 ...

omnicommerce.ai

https://omnicommerce.ai > ko-kr > resources > eibeulri...

에이블리, AI를 활용해 구매 전환율을 높인 초개인화 레시피

2022. 10. 7. — 에이블리는 이러한 목표 달성을 위해 개인화된 쇼핑 경험을 제공하여 구매 여정을 최적화하고자 했어요. 이 과정에서 AI 기반 초개인화 추천 알고리즘의 ...

그린랩스/원티드랩

그린랩스 인공지능

 인공지능신문
https://www.aitimes.kr › news › articleView

농사일, 데이터와 AI에 맡기는...애그테크 스타트업 '그린랩스' ...
2021. 12. 27. — SK스퀘어는 인공지능(AI) 데이터 기반 국내 최대 농업 혁신기업 그린랩스(Green Labs, 각자대표 신상훈·최성우·안동현)에 약 350억원을 투자했다고 27일 ...

 그린랩스
https://greenlabs.co.kr › 뉴스레터 › 미래-농업의-상...

미래 농업의 상징, 정밀농업 튜아보기
2022. 12. 23. — 정밀농업은 위성 기술이나 사물인터넷(IoT), 인공지능(AI), 드론, 빅데이터 기술과 같은 정보통신기술(ICT)을 활용해 토양과 작물의 특성에 맞는 농자재를 ...

 스마트 시티 솔루션 마켓
http://smartcitysolutionmarket.com › slutnCmpnyView

주식회사 그린랩스
생산관리에만 국한된 기존 스마트팜 시스템과 차별화를 둔 그린랩스는 사물인터넷, 빅데이터, 인공지능 기술을 기반으로 농작물 생산관리, 유통 관리, 판로연결에 ...

 특허법인 비엘티
https://blt.kr › inno

그린랩스 - 6차 산업혁명의 첨병, 스마트팜 : 혁신칼럼
2023. 5. 4. — 결국 수 많은 '4차' 산업혁명 아이템 중 인공지능(AI) 아이템만 남았고, 나머지 아이템들은 퇴색했다. '산업혁명'이라는 것은 후대에서 우리를 평가하면서, ...

 서울경제
https://www.sedaily.com › NewsView

그린랩스, 축산 AI '파이프트리 스마트팜'에 투자
2021. 5. 4. — 파이프트리는 양계농가 질병예방 및 농장관리 인공지능(AI) 기술을 중심으로 한 스마트팜 시스템 제공 기업이다. 자체 개발한 센서를 축사에 부착한 후 ...

원티드랩 인공지능

원티드 AI 면접코칭
원티드 AI 면접코칭. 면접 예상 질문부터 답변 피드백까지. 채용공고 분석하기. 샘플 포지션 링크를 복사해 보세요! 원티드랩. 프로젝트 오너 포지션. 링크 복사.

 원티드
https://asr.wanted.co.kr

Wanted AI - 원티드
원티드랩 AI팀은 채용중. 머신러닝 리서치 사이언티스트. 주요업무 • 딥러닝 기반의 매칭 알고리즘과 이력서/채용공고 분석 ...

 Wanted Lab, Inc.
https://blog.wantedlab.com › service-post

원티드 AI 면접코칭 (ChatGPT)
2023. 3. 13. — 바로 <원티드 AI 면접코칭>! 원티드가 ChatGPT 기술을 활용하여 예상 면접 질문은 물론, 답변에 대한 피드백까지 받아볼 수 있습니다. 이제 더 빠르게 ...

 지디넷코리아
https://zdnet.co.kr › view

원티드 AI 면접코칭 받아보니..."동료보다 낫네"
2023. 3. 17. — 채용 플랫폼 원티드랩이 이달 초 선보인 'AI 면접 코칭 서비스'가 그중 하나다. 면접 코칭 서비스에는 언어모델 GPT-3이 활용됐다. 원티드랩, 챗GPT 기술 ...

 머니투데이
https://news.mt.co.kr › mtview

"연봉 1억 달성' 로드맵 만들어준다" 원티드랩 AI서비스 출시
2023. 5. 11. — AI(인공지능) 기반 HR(인적자원관리) 테크 기업 원티드랩은 11일 AI가 이용자의 커리어 설계를 도와주는 '커리어맵'을 출시했다고 밝혔다.

Data & ML & LLM & AI 관련 직군

Data Analyst

데이터를 수집, 처리, 분석하여 비즈니스 의사 결정을 지원하는 전문가.
1주로 통계, 데이터 시각화 도구 및 SQL과 같은 쿼리 언어를 사용

Data Scientist

데이터로부터 숨겨진 패턴이나 정보를 발견하고 예측 모델을 구축하는 전문가.
통계, 머신러닝, 프로그래밍 기술을 사용

Data Architect

기업의 데이터 아키텍처와 관련된 전체적인 설계 및 구조를 담당하는 전문가.
데이터베이스 설계, 데이터 모델링 및 시스템 통합에 관련된 업무를 수행

Data Engineer

대규모 데이터 파이프라인과 관련된 인프라를 구축하고 유지 관리하는 전문가.
ETL 작업, 데이터베이스 관리 및 빅 데이터 기술을 주로 다룸

ML Engineer

머신러닝 모델을 설계, 훈련, 배포하는 전문가.
데이터 과학자와 함께 작업하여 모델을 실제 서비스 환경에 적용

AI Engineer

인공지능 솔루션을 개발하고 구현하는 전문가. ML Engineer와 비슷하지만,
보다 광범위한 AI 영역을 포함하여 딥러닝, 로봇 과학, 음성 인식 등의 기술에 초점

Computer Vision Engineer

이미지 및 비디오 데이터에서 정보를 추출하는 알고리즘과 기술을 개발하는 전문가.

NLP Engineer

자연어 처리 기술을 활용하여 텍스트 데이터에서 정보를 추출하고,
텍스트와 컴퓨터 간의 상호 작용을 개선하는 전문가.

MLOps Engineer

머신러닝의 운영 단계와 관련된 업무를 담당하는 전문가.
모델의 배포, 모니터링, 버전 관리 등을 포함하여 ML 시스템의 전체 수명 주기를 관리

Prompt Engineer (Large Language Model)

큰 언어 모델을 위한 프롬프트를 디자인하고 최적화하는 전문가.
사용자와 모델 간의 상호 작용을 개선하고 효과적인 정보 제공을 지원

진로 탐색 / 취업 준비 노하우

진로 탐색 아이디어

- 학교별 채용공고 사이트 활용
- 구인구직정보 사이트 활용
- 금융(증권/자산운용)은 금융투자협회에서
- 각 산업은 산업별 협회 활용하기
- 인적 네트워크 / 헤드헌터 활용해서 좋은 채용 방법 확인

끝이 다른 시작
JOBKOREA



진로 탐색 / 취업 준비 노하우

indeed Find jobs Company reviews Find salaries

What mlops engineer Where City, state, zip code, or "remote" Search

Date posted Remote Pay Job type Encouraged to apply Location Company

Posted by Experience level Education

Upload your resume - Let employers find you

mlops engineer jobs

Sort by: relevance - date 388 jobs

Senior Machine Learning Engineer, Foundation - Autonomous Vehicles...
NVIDIA
Santa Clara, CA
\$176,000 - \$333,500 a year Full-time
Designing our training code to allow for more engineers to use it simultaneously, with more tasks.
Working closely with platform and perception DNN engineers, ...
Posted 10 days ago
View all 2 available locations

MLOps Principal Engineer
Rite Pros
Portland, ME 04101 (Downtown area)
Pay information not provided
Collaborate with data scientists, engineers, and stakeholders to develop, deploy, and maintain machine learning models in a production environment.
Posted 30+ days ago
View all 2 available locations

Machine Learning Engineer-Minneapolis
Polaris Inc.
Plymouth, MN
Pay information not provided Full-time

MLOps Principal Engineer
Rite Pros 565 Congress St Ste 305, Portland, ME 04101
Apply on company site

Full job description

MLOps Principal Engineer with Bachelor's Degree in Computer Science, Computer Information Systems, Information Technology, or a combination of education and experience equating to the U.S. equivalent of a Bachelor's degree in one of the aforementioned subjects.

Job Duties and Responsibilities:

- Collaborate with data scientists, engineers, and stakeholders to develop, deploy, and maintain machine learning models in a production environment.
- Design, build, and maintain scalable and robust data pipelines to support machine learning workflows.
- Implement and manage version control, continuous integration, and continuous deployment (CI/CD) systems for machine learning models and related software components.
- Monitor the performance and health of deployed machine learning models, making necessary adjustments and improvements to ensure optimal performance and reliability.
- Develop and maintain tools and processes to automate and streamline machine learning model deployment, monitoring, and management.
- Ensure data privacy, security, and compliance with relevant regulations and best practices.
- Troubleshoot and resolve issues related to machine learning model deployment and infrastructure.
- Stay up-to-date on industry trends, emerging technologies, and best practices in MLOps, and contribute to the continuous improvement of the team's processes and tools.
- Provide technical guidance and support to data scientists and other team members on best practices for model deployment, monitoring, and management.
- Document and communicate MLOps processes, guidelines, and procedures to ensure consistency and knowledge sharing across the organization.

Technologies Involved / Skills required for the position

Messages

진로 탐색 / 취업 준비 노하우

진로 준비 계획 아이디어 02 (Salary 파악)

- Google에 “MLOps Engineer Salary” 검색
- 평균 연봉 혹은 연봉의 Range 파악

Google mlops engineer salary

이미지 뉴스 US 동영상 London UK 쇼핑 도서 지도

검색결과 약 229,000개 (0.41초)

Glassdoor
https://www.glassdoor.com › Salaries › mlops-engineer-s...

Salary: MLOps Engineer (September, 2023)
2023. 5. 26. — The average salary for a MLOps Engineer is **\$124,711 per year** in United States. Salaries estimates are based on 22 salaries submitted anonymously ...

관련 질문 :

- How much does an MLOps engineer make?
- Are MLOps engineers in demand?
- What does an MLOps engineer do?
- What is the salary of MLOps analyst?

사용자 의견

Glassdoor
https://www.glassdoor.ca › Salaries › us-mlops-engine...

Salary: MLOps Engineer (September, 2023)
MLOps Engineer Salaries ... The average salary for MLOps Engineer is **US\$124,724 per year** in the United States. The average additional cash compensation for a ...

figr.com
https://6figr.com › salary › mlops-s

Mlops Salaries 2023 | \$175k-\$281k
Employees who knows Mlops earn an average of \$211k, mostly ranging from \$175k per year to **\$280k per year** based on 4 profiles. The top 10% of employees earn more ...

진로 탐색 / 취업 준비 노하우

진로 준비 계획 아이디어 03 (국내 AI 일자리 파악 및 지원)

- JD 기반 필수역량, 우대역량 정리
 - 네카라쿠배 / 당토 / 직야 / 몰두센 / 마에그원
 - 기타 원하는 산업 분야 스타트업
- **우선순위 정해서 역량 키우기**
- **JD와 관련 본인의 핵심 역량 최대화한 이력서 작성**

Google mlops engineer salary

이미지 뉴스 US 동영상 London UK 쇼핑 도서 지도

검색결과 약 229,000개 (0.41초)

Glassdoor
https://www.glassdoor.com › Salaries › mlops-engineer-s...

Salary: MLOps Engineer (September, 2023)
2023. 5. 26. — The average salary for a MLOps Engineer is **\$124,711 per year** in United States. Salaries estimates are based on 22 salaries submitted anonymously ...

관련 질문 :

- How much does an MLOps engineer make? ▾
- Are MLOps engineers in demand? ▾
- What does an MLOps engineer do? ▾
- What is the salary of MLOps analyst? ▾

사용자 의견

Glassdoor
https://www.glassdoor.ca › Salaries › us-mlops-engine...

Salary: MLOps Engineer (September, 2023)
MLOps Engineer Salaries ... The average salary for MLOps Engineer is **US\$124,724 per year** in the United States. The average additional cash compensation for a ...

figr.com
https://6figr.com › salary › mlops-s

Mlops Salaries 2023 | \$175k-\$281k
Employees who knows Mlops earn an average of \$211k, mostly ranging from \$175k per year to **\$280k per year** based on 4 profiles. The top 10% of employees earn more ...

면접 준비 노하우

면접 준비 때 꼭 챙겨야할 것!

1. 1분 자기 소개 핵심만 간추려서 외우기
2. 기술 면접은 구글링을 해서 총 취합된 정보들을 미리 다 외우기
3. 면접관도 면접보듯이 잘 살펴보기 (같이 일할 사람들이 좋은지 미리 파악할 수 있는 유일한 기회)
4. 면접 때 질문 많이하기 (같이 일할 사람들이 좋다면, 열정 가득담아 보여주기)

인터뷰 준비 및 인사팀 인적성 질문 대비 노하우

- 인터뷰 이전 준비 과정 - [Notion \[Interview\] 01. 준비 노하우](#)
- 인터뷰 이전 인적성 예시 질문 답변해보기 - [Notion \[Interview\] 02. Mindset Q&A 질문](#)
- 인터뷰 이전 인적성 예시 질문 의도 및 추천 답변 확인 - [Notion \[Interview\] 03. Mindset Q&A 질문 의도 및 추천 답변](#)



기말 프로젝트 관련 공지

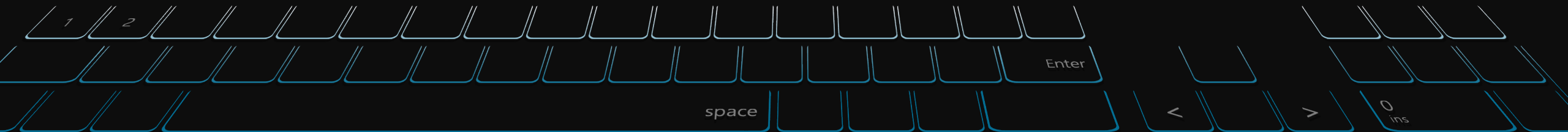
Info about Final Term Project

기말 프로젝트 안내

1. 팀 구성 마감
 - 마감일: 2026년 5월 9일 (토요일 밤 12:00)까지
2. 기획서 초안 제출
 - 제출 마감: 2026년 5월 15일 (금요일 밤 12:00)까지
3. 기획서 예시
 - 예시 문서 링크: [기획서 예시 보러가기](#)

Q&A

질의응답 (5분)



끝. 감사합니다.

수업 듣느라 수고하셨습니다.

